

尚硅谷大数据技术之 SparkCore

（作者：尚硅谷研究院）

版本：V5.0

第 1 章 RDD 概述

1.1 什么是 RDD

RDD（Resilient Distributed Dataset）叫做弹性分布式数据集，是 Spark 中最基本的数据抽象。

代码中是一个抽象类，它代表一个弹性的、不可变、可分区、里面的元素可并行计算的集合。

RDD 类比工厂生产。



• 1.2 RDD 五大特性



RDD特性



* Internally, each RDD is characterized by five main properties:

- * - A list of partitions
- * - A function for computing each split
- * - A list of dependencies on other RDDs
- * - Optionally, a Partitioner for key-value RDDs (e.g. to say that the RDD is hash-partitioned)
- * - Optionally, a list of preferred locations to compute each split on (e.g. block locations for an HDFS file)

(1) 一组分区 (Partition)，即是数据集的基本组成单位，标记数据是哪个分区的：

```
protected def getPartitions: Array[Partition]
```

(2) 一个计算每个分区的函数：

```
def compute(split: Partition, context: TaskContext): Iterator[T]
```

(3) RDD之间的依赖关系：

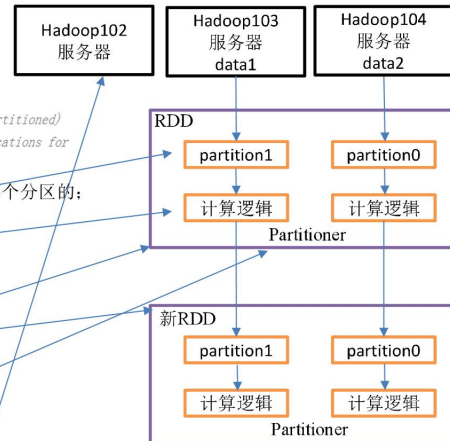
```
protected def getDependencies: Seq[Dependency[_]] = deps
```

(4) 一个Partitioner，即RDD的分片函数；控制分区的数据流向(键值对)

```
val partitioner: scala.Option[org.apache.spark.Partitioner]
```

(5) 一个列表，存储存取每个Partition的优先位置 (preferred location)。如果节点和分区个数不对应优先把分区设置在哪个节点上。移动数据不如移动计算，除非资源不够。

```
protected def getPreferredLocations(split: Partition): scala.Seq[String]
```



让天下没有难学的技术

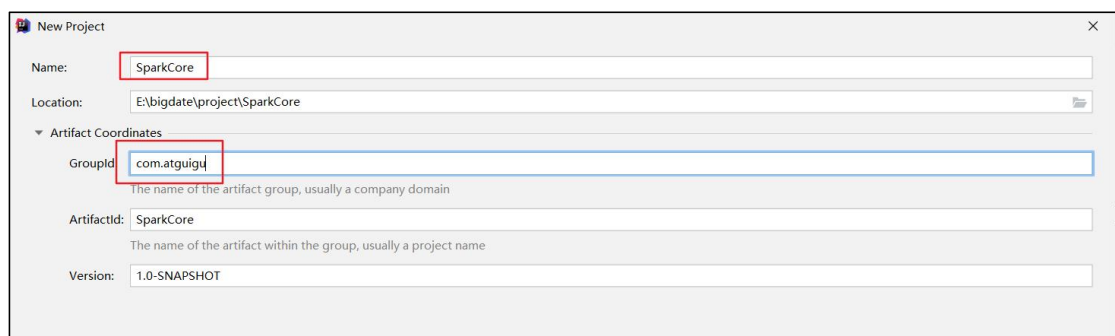
第 2 章 RDD 编程

2.1 RDD 的创建

在 Spark 中创建 RDD 的创建方式可以分为三种：从集合中创建 RDD、从外部存储创建 RDD、从其他 RDD 创建。

2.1.1 IDEA 环境准备

1) 创建一个 maven 工程，工程名称叫 SparkCore



2) 创建包名：com.atguigu.createrdd

3) 在 pom 文件中添加 spark-core 的依赖

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
```

```
<version>3.3.1</version>
</dependency>
</dependencies>
```

4) 如果不希望运行时打印大量日志，可以在 resources 文件夹中添加 `log4j2.properties` 文件，并添加日志配置信息

```
# Set everything to be logged to the console
rootLogger.level = ERROR
rootLogger.appenderRef.stdout.ref = console

# In the pattern layout configuration below, we specify an explicit '%ex' conversion
# pattern for logging Throwables. If this was omitted, then (by default) Log4J would
# implicitly add an '%xEx' conversion pattern which logs stacktraces with additional
# class packaging information. That extra information can sometimes add a substantial
# performance overhead, so we disable it in our default logging config.
# For more information, see SPARK-39361.
appender.console.type = Console
appender.console.name = console
appender.console.target = SYSTEM_ERR
appender.console.layout.type = PatternLayout
appender.console.layout.pattern = %d{yy/MM/dd HH:mm:ss} %p %c{1}: %m%n%ex

# Set the default spark-shell/spark-sql log level to WARN. When running the
# spark-shell/spark-sql, the log level for these classes is used to overwrite
# the root logger's log level, so that the user can have different defaults
# for the shell and regular Spark apps.
logger.repl.name = org.apache.spark.repl.Main
logger.repl.level = warn
logger.thriftserver.name = org.apache.spark.sql.hive.thriftserver.SparkSQLCLIDriver
logger.thriftserver.level = warn

# Settings to quiet third party logs that are too verbose
logger.jetty1.name = org.sparkproject.jetty
logger.jetty1.level = warn
logger.jetty2.name = org.sparkproject.jetty.util.component.AbstractLifeCycle
logger.jetty2.level = error
logger.replExprTyper.name = org.apache.spark.repl.SparkIMain$ExprTyper
logger.replExprTyper.level = info
logger.replSparkILoopInterpreter.name = org.apache.spark.repl.SparkILoop$SparkILoopInterpreter
logger.replSparkILoopInterpreter.level = info
logger.parquet1.name = org.apache.parquet
logger.parquet1.level = error
logger.parquet2.name = parquet
logger.parquet2.level = error

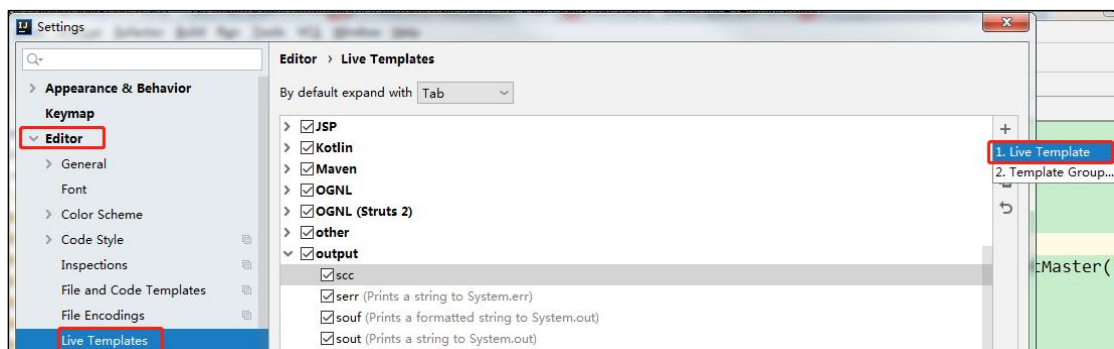
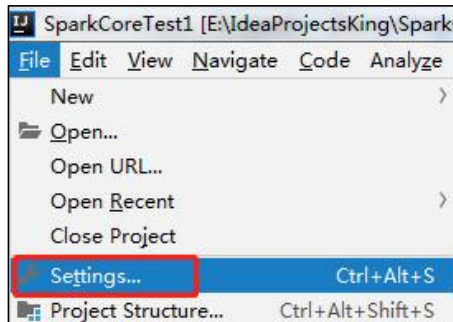
# SPARK-9183: Settings to avoid annoying messages when looking up nonexistent UDFs in
SparkSQL with Hive support
logger.RetryingHMSHandler.name = org.apache.hadoop.hive.metastore.RetryingHMSHandler
logger.RetryingHMSHandler.level = fatal
logger.FunctionRegistry.name = org.apache.hadoop.hive.ql.exec.FunctionRegistry
logger.FunctionRegistry.level = error

# For deploying Spark ThriftServer
# SPARK-34128: Suppress undesirable TTransportException warnings involved in THRIFT-4805
```

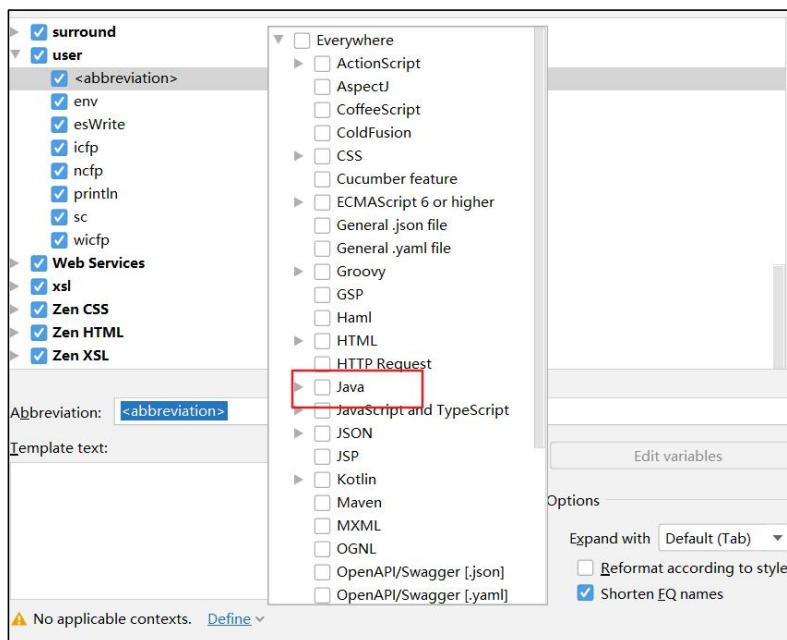
```
appender.console.filter.1.type = RegexFilter  
appender.console.filter.1.regex = .*Thrift error occurred during processing of message.*  
appender.console.filter.1.onMatch = deny appender.console.filter.1.onMismatch = neutral
```

2.1.2 创建 IDEA 快捷键

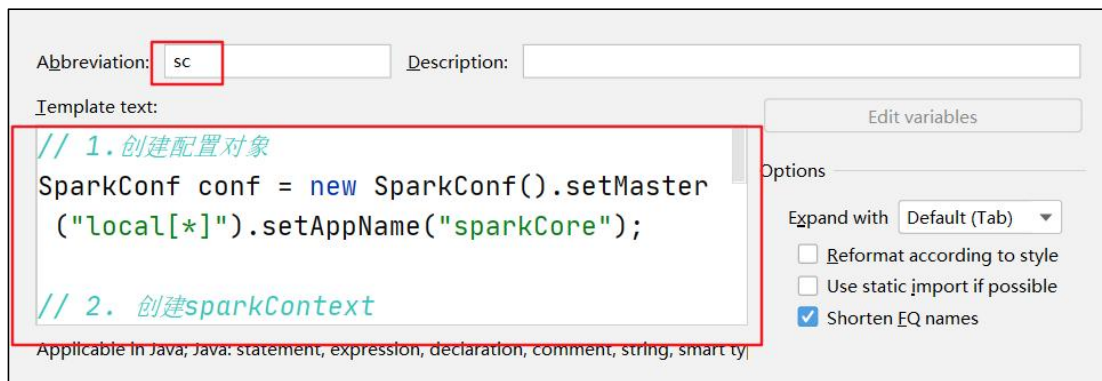
1) 点击 File->Settings...->Editor->Live Templates->output->Live Template



2) 点击左下角的 Define->选择 JAVA



3) 在 Abbreviation 中输入快捷键名称 sc，在 Template text 中填写，输入快捷键后生成的内容。



```
// 1.创建配置对象
SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

// 2. 创建 sparkContext
JavaSparkContext sc = new JavaSparkContext(conf);

// 3. 编写代码

// 4. 关闭 sc
sc.stop();
```

2.1.3 异常处理

如果程序代码出现以下错误：

```
Exception in thread "main" java.lang.NoClassDefFoundError: Create breakpoint :
org/apache/hadoop/shaded/org/apache/commons/configuration2/Configuration
    at org.apache.hadoop.metrics2.lib.DefaultMetricsSystem.<init>(DefaultMetricsSystem.java:43)
    at org.apache.hadoop.metrics2.lib.DefaultMetricsSystem.<clinit>(DefaultMetricsSystem.java:41)
    at org.apache.hadoop.security.UserGroupInformation$UgiMetrics.create(UserGroupInformation.java:149)
    at org.apache.hadoop.security.UserGroupInformation.<clinit>(UserGroupInformation.java:265)
    at org.apache.spark.util.Utils$.anonfun$getCurrentUserName$1(Utils.scala:2561)
    at scala.Option.getOrElse(Option.scala:189)
    at org.apache.spark.util.Utils$.getCurrentUserName(Utils.scala:2561)
    at org.apache.spark.SparkContext.<init>(SparkContext.scala:316)
    at org.apache.spark.api.java.JavaSparkContext.<init>(JavaSparkContext.scala:58)
    at org.codehaus.mojo.exec.ExecMain.launch(ExecMain.java:11)
    at org.codehaus.mojo.exec.ExecMain.main(ExecMain.java:11)
Caused by: java.lang.ClassNotFoundException: Create breakpoint : org.apache.hadoop.shaded.org.apache.commons.configuration2.Configuration
    at java.net.URLClassLoader.findClass(URLClassLoader.java:381)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:424)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:331)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:357)
    ... 10 more
```

请采用资料中的 hadoop 环境。

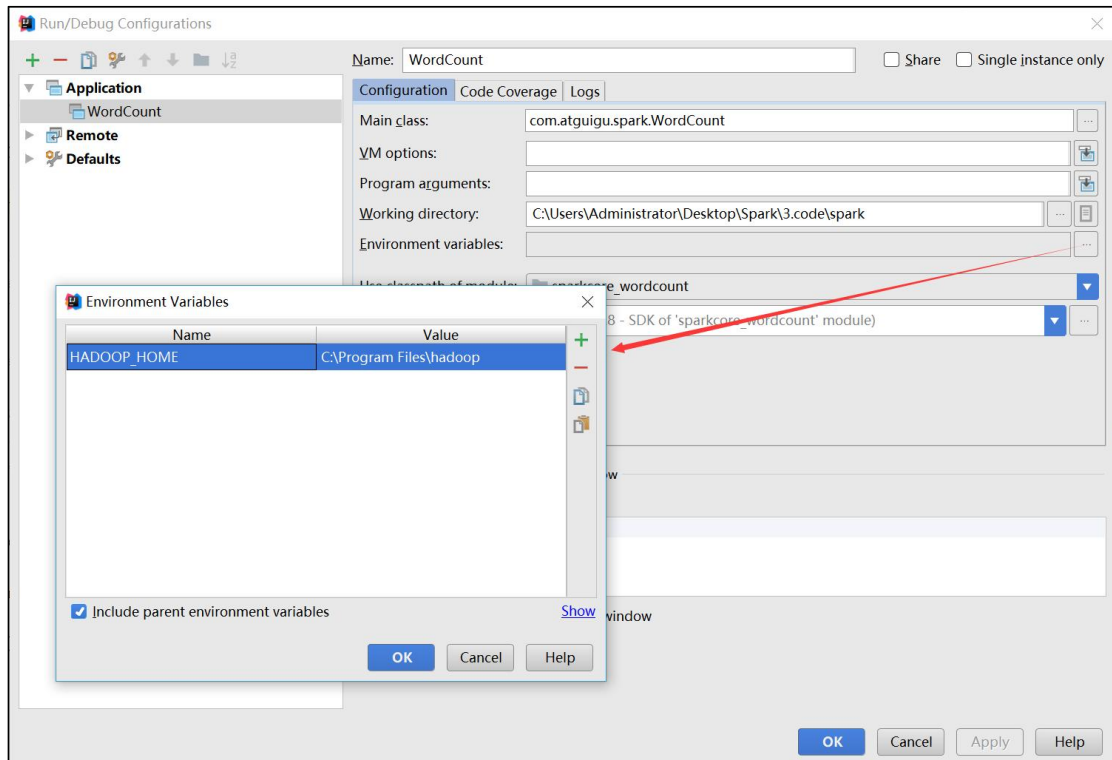
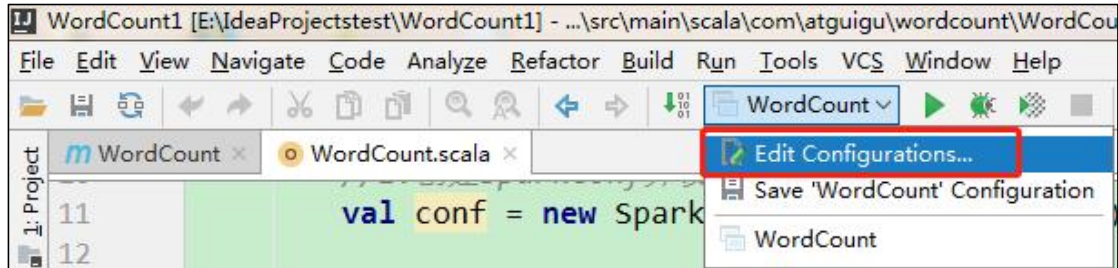
如果本机操作系统是 Windows，在程序中使用了 Hadoop 相关的东西，比如写入文件到 HDFS，则会遇到如下异常：

```
2017-09-14 16:08:34,907 ERROR --- [main] org.apache.hadoop.util.Shell(line:303) : Failed to locate the winutils binary in the hadoop binary path
java.io.IOException: Could not locate executable null\bin\winutils.exe in the Hadoop binaries.
    at org.apache.hadoop.util.Shell.getQualifiedBinPath(Shell.java:278)
    at org.apache.hadoop.util.Shell.getWinUtilsPath(Shell.java:300)
    at org.apache.hadoop.util.Shell.<clinit>(Shell.java:293)
    at org.apache.hadoop.util.StringUtils.<clinit>(StringUtils.java:76)
    at org.apache.hadoop.conf.Configuration.getTrimmedStrings(Configuration.java:1546)
    at org.apache.hadoop.hdfs.DFSClient.<init>(DFSClient.java:519)
    at org.apache.hadoop.hdfs.DFSClient.<init>(DFSClient.java:453)
    at org.apache.hadoop.hdfs.DistributedFileSystem.initialize(DistributedFileSystem.java:136)
    at org.apache.hadoop.fs.FileSystem.createFileSystem(FileSystem.java:2433)
```


出现这个问题的原因，并不是程序的错误，而是用到了 Hadoop 相关的服务，解决办法

1) 配置 HADOOP_HOME 环境变量

2) 在 IDEA 中配置 Run Configuration，添加 HADOOP_HOME 变量

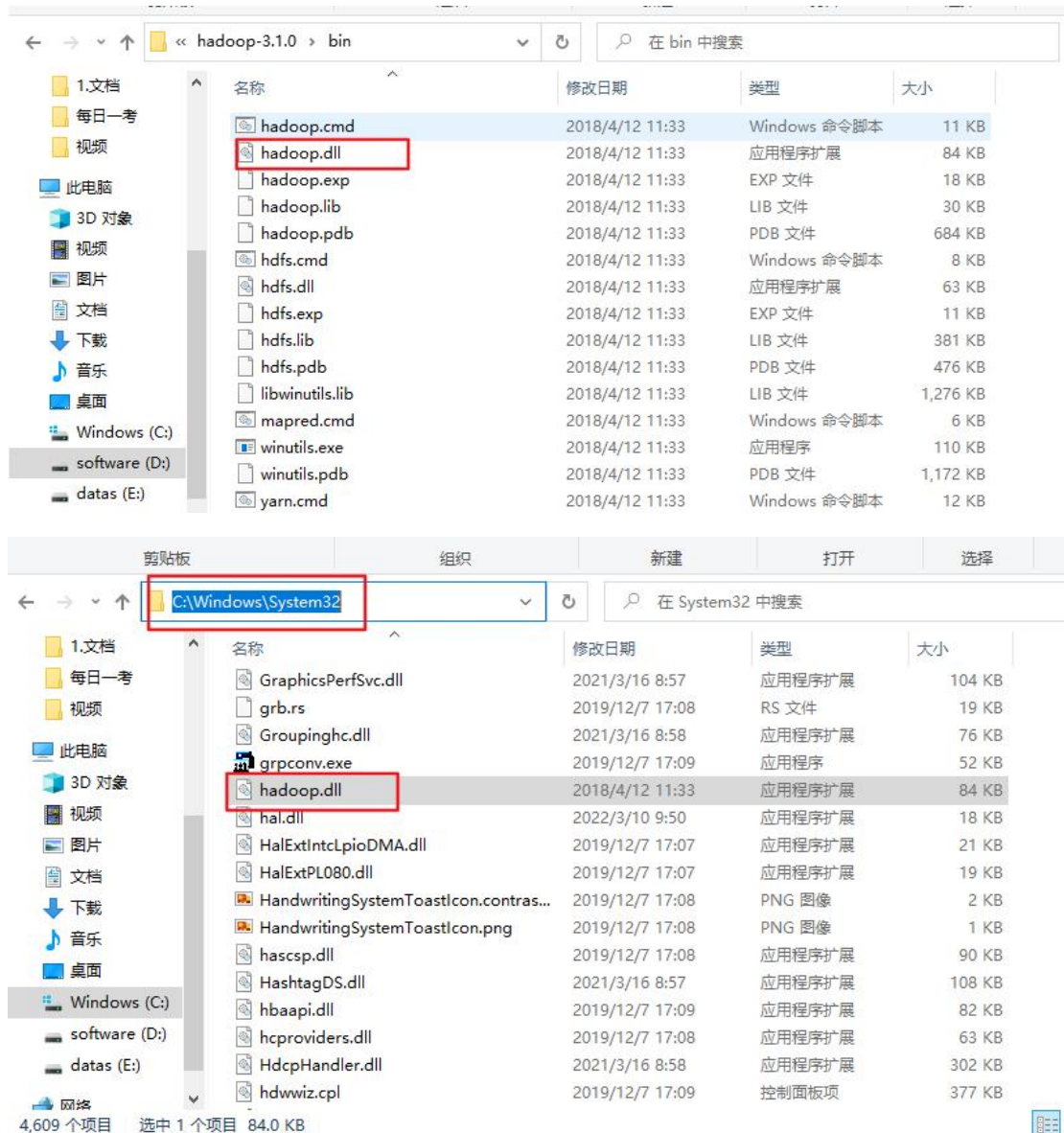


如果出现这个问题，是因为 windows 上面的 hadoop 权限不够。

```
Exception in thread "main" java.lang.UnsatisfiedLinkError:
org.apache.hadoop.io.nativeio.NativeIO$Windows.access0(Ljava/lang
/String;I)Z
    at
org.apache.hadoop.io.nativeio.NativeIO$Windows.access0(Native
Method)
    at
org.apache.hadoop.io.nativeio.NativeIO$Windows.access(NativeIO.ja
va:645)
    at org.apache.hadoop.fs.FileUtil.canRead(FileUtil.java:1230)
    at org.apache.hadoop.fs.FileUtil.list(FileUtil.java:1435)
    at
org.apache.hadoop.fs.RawLocalFileSystem.listStatus(RawLocalFileSy
```

```
stem.java:493)
    at
org.apache.hadoop.fs.FileSystem.listStatus(FileSystem.java:1868)
    at
org.apache.hadoop.fs.FileSystem.listStatus(FileSystem.java:1910)
    at
org.apache.hadoop.fs.FileSystem$4.<init>(FileSystem.java:2072)
```

解决方法是把安装在 windows 上面的 hadoop 的 bin 文件夹中的 hadoop.dll 复制到 C:\Windows\System32 文件夹中。



2.1.4 从集合中创建

1) 从集合中创建 RDD: parallelize

```
package com.atguigu.createrdd;

import org.apache.spark.SparkConf;
```

```
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.Arrays;
import java.util.List;

public class Test01_List {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<String> stringRDD = sc.parallelize(Arrays.asList("hello", "spark"));

        List<String> result = stringRDD.collect();

        for (String s : result) {
            System.out.println(s);
        }

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.1.5 从外部存储系统的数据集创建

由外部存储系统的数据集创建 RDD 包括：本地的文件系统，还有所有 Hadoop 支持的数据集，比如 HDFS、HBase 等。

1) 数据准备

在新建的 SparkCore 项目名称上右键=》新建 input 文件夹=》在 input 文件夹上右键=》分别新建 1.txt 和 2.txt。每个文件里面准备一些 word 单词。

2) 创建 RDD

```
package com.atguigu.createrdd;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.List;

public class Test02_File {
    public static void main(String[] args) {
        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");
```



```
// 2. 创建 sparkContext
JavaSparkContext sc = new JavaSparkContext(conf);

// 3. 编写代码
JavaRDD<String> lineRDD = sc.textFile("input");

List<String> result = lineRDD.collect();

for (String s : result) {
    System.out.println(s);
}

// 4. 关闭 sc
sc.stop();
}
```

2.1.6 从其他 RDD 创建

主要是通过一个 RDD 运算完后，再产生新的 RDD。

详见 2.3 节

2.2 分区规则

2.2.1 从集合创建 RDD

1) 创建一个包名：com.atguigu.partition

2) 代码验证

```
package com.atguigu.partition;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.Arrays;

public class Test01_ListPartition {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        // 默认环境的核数
        // 可以手动填写参数控制分区的个数
        JavaRDD<String> stringRDD = sc.parallelize(Arrays.asList("hello", "spark", "hello", "spark", "hello"),2);
    }
}
```

```
// 数据分区的情况
// 0 => 1,2 1 => 3,4,5
// 利用整数除机制 左闭右开
// 0 => start 0*5/2 end 1*5/2
// 1 => start 1*5/2 end 2*5/2
stringRDD.saveAsTextFile("output");

// 4. 关闭 sc
sc.stop();
}
}
```

2.2.2 从文件创建 RDD

1) 分区测试

```
package com.atguigu.partition;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.List;

public class Test02_FilePartition {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        // 默认填写的最小分区数 2 和环境的核数取小的值 一般为 2
        JavaRDD<String> lineRDD = sc.textFile("input/1.txt");

        // 具体的分区个数需要经过公式计算
        // 首先获取文件的总长度 totalSize
        // 计算平均长度 goalSize = totalSize / numSplits
        // 获取块大小 128M
        // 计算切分大小 splitSize = Math.max(minSize, Math.min(goalSize, blockSize));
        // 最后使用 splitSize 按照 1.1 倍原则切分整个文件 得到几个分区就是几个分区

        // 实际开发中 只需要看文件总大小 / 填写的分区数 和块大小比较 谁小拿谁进行切分
        lineRDD.saveAsTextFile("output");

        // 数据会分配到哪个分区
        // 如果切分的位置位于一行的中间 会在当前分区读完一整行数据

        // 0 -> 1,2 1 -> 3 2 -> 4 3 -> 空

        // 4. 关闭 sc
        sc.stop();
    }
}
```

```
}  
}
```

2) 分区源码

注意: `getSplits` 文件返回的是切片规划, 真正读取是在 `compute` 方法中创建

`LineRecordReader` 读取的, 有两个关键变量: `start = split.getStart()` `end = start + split.getLength`

①分区数量的计算方式:

`totalSize = 10`

`goalSize = 10 / 3 = 3(byte)` 表示每个分区存储 3 字节的数据

分区数 = `totalSize / goalSize = 10 / 3 => 3,3,4`

4 字节大于 3 字节的 1.1 倍, 符合 `hadoop` 切片 1.1 倍的策略, 因此会多创建一个分区, 即一共有 4 个分区 3,3,3,1

②Spark 读取文件, 采用的是 `hadoop` 的方式读取, 所以一行一行读取, 跟字节数没有关系

③数据读取位置计算是以偏移量为单位来进行计算的。

2.3 Transformation 转换算子

2.3.1 Value 类型

1) 创建包名: `com.atguigu.value`

2.3.1.1 map()映射

参数 `f` 是一个函数可以写作匿名子类, 它可以接收一个参数。当某个 RDD 执行 `map` 方法时, 会遍历该 RDD 中的每一个数据项, 并依次应用 `f` 函数, 从而产生一个新的 RDD。即, 这个新 RDD 中的每一个元素都是原来 RDD 中每一个元素依次应用 `f` 函数而得到的。

1) 具体实现

```
package com.atguigu.value;  
  
import org.apache.spark.SparkConf;  
import org.apache.spark.api.java.JavaRDD;  
import org.apache.spark.api.java.JavaSparkContext;  
import org.apache.spark.api.java.function.Function;  
  
public class Test01_Map {  
    public static void main(String[] args) {  
  
        // 1.创建配置对象  
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");  
  
        // 2. 创建 sparkContext  
        JavaSparkContext sc = new JavaSparkContext(conf);
```

```
// 3. 编写代码
JavaRDD<String> lineRDD = sc.textFile("input/1.txt");

// 需求:每行结尾拼接\
// 两种写法 lambda 表达式写法(匿名函数)
JavaRDD<String> mapRDD = lineRDD.map(s -> s + "\\");

// 匿名函数写法
JavaRDD<String> mapRDD1 = lineRDD.map(new Function<String, String>() {
    @Override
    public String call(String v1) throws Exception {
        return v1 + "\\";
    }
});

for (String s : mapRDD.collect()) {
    System.out.println(s);
}

// 输出数据的函数写法
mapRDD1.collect().foreach(a -> System.out.println(a));
mapRDD1.collect().foreach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
```

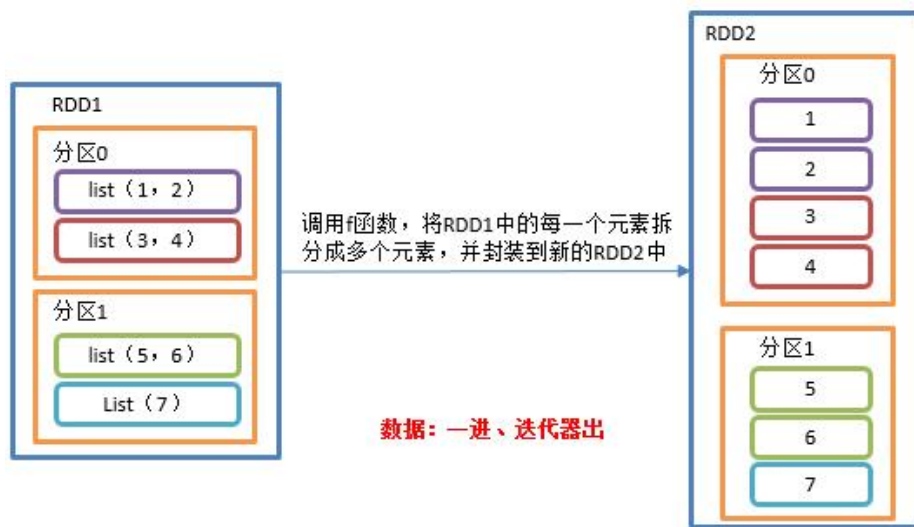
2.3.1.2 flatMap()扁平化

1) 功能说明

与 map 操作类似，将 RDD 中的每一个元素通过应用 f 函数依次转换为新的元素，并封装到 RDD 中。

区别：在 flatMap 操作中，f 函数的返回值是一个集合，并且会将每一个该集合中的元素拆分出来放到新的 RDD 中。

2) 需求说明：创建一个集合，集合里面存储的还是子集合，把所有子集合中数据取出放入到一个大的集合中。



4) 具体实现:

```
package com.atguigu.value;

import org.apache.commons.collections.ListUtils;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

public class Test02_FlatMap {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        ArrayList<List<String>> arrayLists = new ArrayList<>();

        arrayLists.add(Arrays.asList("1","2","3"));
        arrayLists.add(Arrays.asList("4","5","6"));

        JavaRDD<List<String>> listJavaRDD = sc.parallelize(arrayLists,2);
        // 对于集合嵌套的 RDD 可以将元素打散
        // 泛型为打散之后的元素类型
        JavaRDD<String> stringJavaRDD = listJavaRDD.flatMap(new
        FlatMapFunction<List<String>, String>() {
            @Override
            public Iterator<String> call(List<String> strings) throws Exception {
```



```
        return strings.iterator();
    }
});

stringJavaRDD.collect().foreach(System.out::println);

// 通常情况下需要自己将元素转换为集合
JavaRDD<String> lineRDD = sc.textFile("input/2.txt");

JavaRDD<String> stringJavaRDD1 = lineRDD.flatMap(new FlatMapFunction<String,
String>() {
    @Override
    public Iterator<String> call(String s) throws Exception {
        String[] s1 = s.split(" ");
        return Arrays.asList(s1).iterator();
    }
});

stringJavaRDD1.collect().foreach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
}
```

2.3.1.3 groupBy()分组

- 1) 功能说明：分组，按照传入函数的返回值进行分组。将相同的 key 对应的值放入一个迭代器。
- 2) 需求说明：创建一个 RDD，按照元素模以 2 的值进行分组。
- 3) 具体实现

```
package com.atguigu.value;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;

import java.util.Arrays;

public class Test03_GroupBy {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),2);
```

```
// 泛型为分组标记的类型
JavaPairRDD<Integer, Iterable<Integer>> groupByRDD = integerJavaRDD.groupBy(new
Function<Integer, Integer>() {
    @Override
    public Integer call(Integer v1) throws Exception {
        return v1 % 2;
    }
});

groupByRDD.collect().foreach(System.out::println);

// 类型可以任意修改
JavaPairRDD<Boolean, Iterable<Integer>> groupByRDD1 = integerJavaRDD.groupBy(new
Function<Integer, Boolean>() {
    @Override
    public Boolean call(Integer v1) throws Exception {
        return v1 % 2 == 0;
    }
});

groupByRDD1.collect().foreach(System.out::println);

Thread.sleep(600000);

// 4. 关闭 sc
sc.stop();
}
```

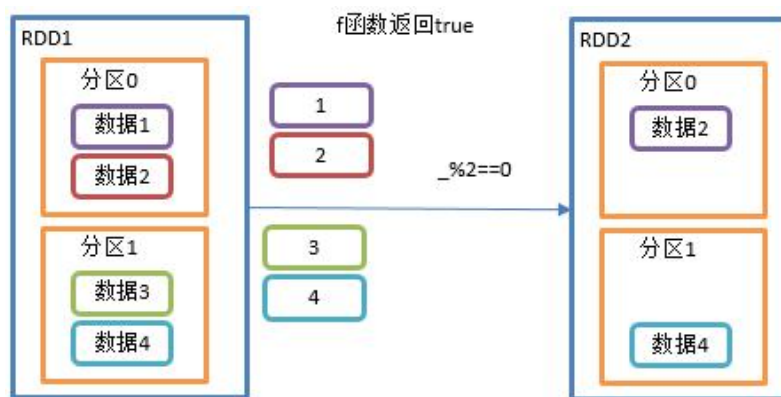
- **groupBy** 会存在 **shuffle** 过程
- **shuffle**: 将不同的分区数据进行打乱重组的过程
- **shuffle** 一定会落盘。可以在 **local** 模式下执行程序，通过 **4040** 看效果。

2.3.1.4 filter()过滤

1) 功能说明

接收一个返回值为布尔类型的函数作为参数。当某个 RDD 调用 **filter** 方法时，会对该 RDD 中每一个元素应用 **f** 函数，如果返回值类型为 **true**，则该元素会被添加到新的 RDD 中。

2) 需求说明：创建一个 RDD，过滤出对 2 取余等于 0 的数据



注意：如果数据2或者数据4，有大量重复数据，是有可能导致数据倾斜的。

3) 代码实现

```
package com.atguigu.value;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;

import java.util.Arrays;

public class Test04_Filter {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),2);

        JavaRDD<Integer> filterRDD = integerJavaRDD.filter(new Function<Integer, Boolean>() {
            @Override
            public Boolean call(Integer v1) throws Exception {
                return v1 % 2 == 0;
            }
        });

        filterRDD.collect().foreach(System.out::println);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.3.1.5 distinct()去重

1) 功能说明：对内部的元素去重，并将去重后的元素放到新的 RDD 中。

2) 代码实现

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网

```
package com.atguigu.value;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.Arrays;

public class Test05_Distinct {
    public static void main(String[] args) {
        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6), 2);

        // 底层使用分布式分组去重 所有速度比较慢,但是不会 OOM
        JavaRDD<Integer> distinct = integerJavaRDD.distinct();

        distinct.collect().foreach(System.out::println);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

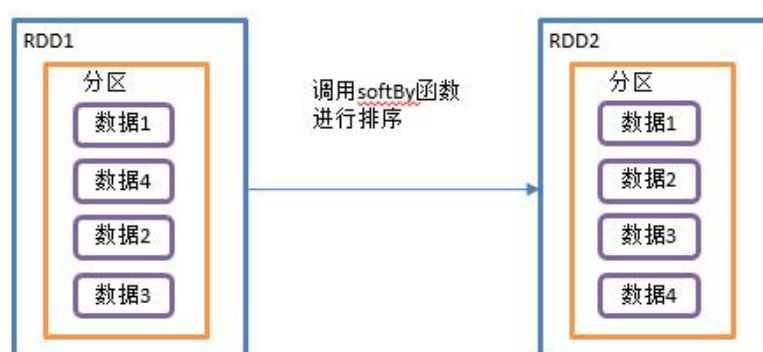
注意：distinct 会存在 shuffle 过程。

2.3.1.6 sortBy()排序

1) 功能说明

该操作用于排序数据。在排序之前，可以将数据通过 f 函数进行处理，之后按照 f 函数处理的结果进行排序，默认为正序排列。排序后新产生的 RDD 的分区数与原 RDD 的分区数一致。Spark 的排序结果是全局有序。

2) 需求说明：创建一个 RDD，按照数字大小分别实现正序和倒序排序



3) 代码实现:

```
package com.atguigu.value;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;

import java.util.Arrays;

public class Test6_SortBy {
    public static void main(String[] args) {
        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(5, 8, 1, 11, 20), 2);

        // (1)泛型为以谁作为标准排序 (2) true 为正序 (3) 排序之后的分区个数
        JavaRDD<Integer> sortByRDD = integerJavaRDD.sortBy(new Function<Integer, Integer>()
        {
            @Override
            public Integer call(Integer v1) throws Exception {
                return v1;
            }
        }, true, 2);

        sortByRDD.collect().foreach(System.out::println);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.3.2 Key-Value 类型

1) 创建包名: com.atguigu.keyvalue

要想使用 Key-Value 类型的算子首先需要使用特定的方法转换为 PairRDD

```
package com.atguigu.keyValue;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

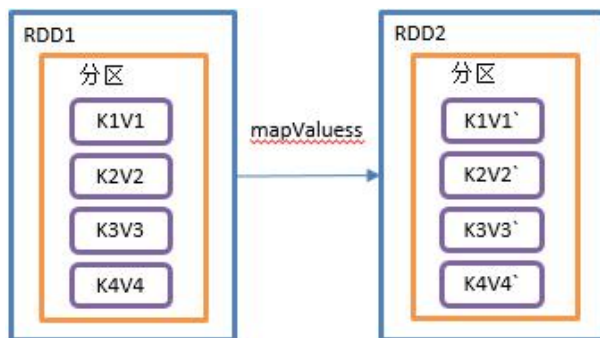
import java.util.Arrays;

public class Test01_pairRDD{
```

```
public static void main(String[] args) {  
  
    // 1.创建配置对象  
    SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");  
  
    // 2. 创建 sparkContext  
    JavaSparkContext sc = new JavaSparkContext(conf);  
  
    // 3. 编写代码  
    JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),2);  
  
    JavaPairRDD<Integer, Integer> pairRDD = integerJavaRDD.mapToPair(new  
PairFunction<Integer, Integer, Integer>() {  
        @Override  
        public Tuple2<Integer, Integer> call(Integer integer) throws Exception {  
            return new Tuple2<>(integer, integer);  
        }  
    });  
  
    pairRDD. collect().forEach(System.out::println);  
  
    // 4. 关闭 sc  
    sc.stop();  
}
```

2.3.2.1 mapValues() 只对 V 进行操作

- 1) 功能说明：针对于(K,V)形式的类型只对 V 进行操作
- 2) 需求说明：创建一个 pairRDD，并将 value 添加字符串"|||"



- 4) 代码实现：

```
package com.atguigu.keyValue;  
  
import org.apache.spark.SparkConf;  
import org.apache.spark.api.java.JavaPairRDD;  
import org.apache.spark.api.java.JavaSparkContext;  
import org.apache.spark.api.java.function.Function;  
import scala.Tuple2;  
  
import java.util.Arrays;
```



```
public class Test02_MapValues {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaPairRDD<String, String> javaPairRDD = sc.parallelizePairs(Arrays.asList(new
        Tuple2<>("k", "v"), new Tuple2<>("k1", "v1"), new Tuple2<>("k2", "v2")));

        // 只修改 value 不修改 key
        JavaPairRDD<String, String> mapValuesRDD = javaPairRDD.mapValues(new
        Function<String, String>() {
            @Override
            public String call(String v1) throws Exception {
                return v1 + "|||";
            }
        });

        mapValuesRDD.collect().foreach(System.out::println);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

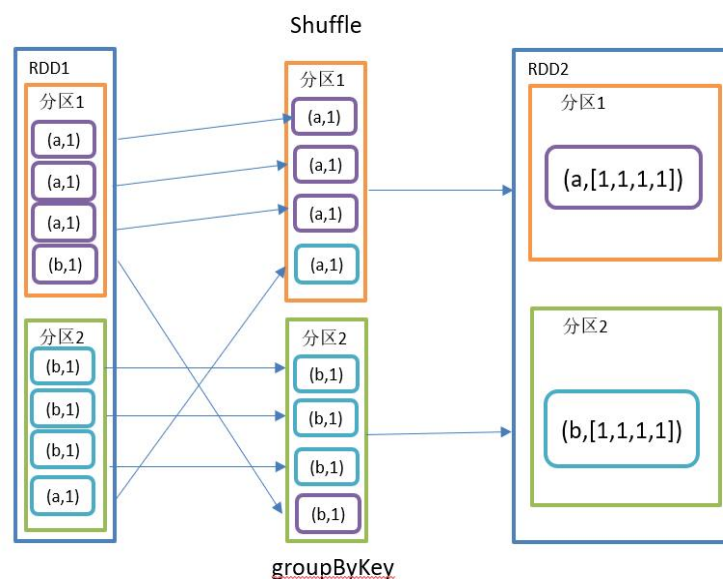
2.3.2.2 groupByKey()按照 K 重新分组

1) 功能说明

groupByKey 对每个 key 进行操作，但只生成一个 seq，并不进行聚合。

该操作可以指定分区器或者分区数（默认使用 HashPartitioner）

2) 需求说明：统计单词出现次数



4) 代码实现:

```
package com.atguigu.keyValue;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.Arrays;

public class Test03_GroupByKey {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<String> integerJavaRDD = sc.parallelize(Arrays.asList("hi","hi","hello","spark"),2);

        // 统计单词出现次数
        JavaPairRDD<String, Integer> pairRDD = integerJavaRDD.mapToPair(new
        PairFunction<String, String, Integer>() {
            @Override
            public Tuple2<String, Integer> call(String s) throws Exception {
                return new Tuple2<>(s, 1);
            }
        });

        // 聚合相同的 key
        JavaPairRDD<String, Iterable<Integer>> groupByKeyRDD = pairRDD.groupByKey();

        // 合并值
        JavaPairRDD<String, Integer> result = groupByKeyRDD.mapValues(new
        Function<Iterable<Integer>, Integer>() {
            @Override
            public Integer call(Iterable<Integer> v1) throws Exception {
                Integer sum = 0;
                for (Integer integer : v1) {
                    sum += integer;
                }
                return sum;
            }
        });

        result.collect().foreach(System.out::println);

        // 4. 关闭 sc
```

```

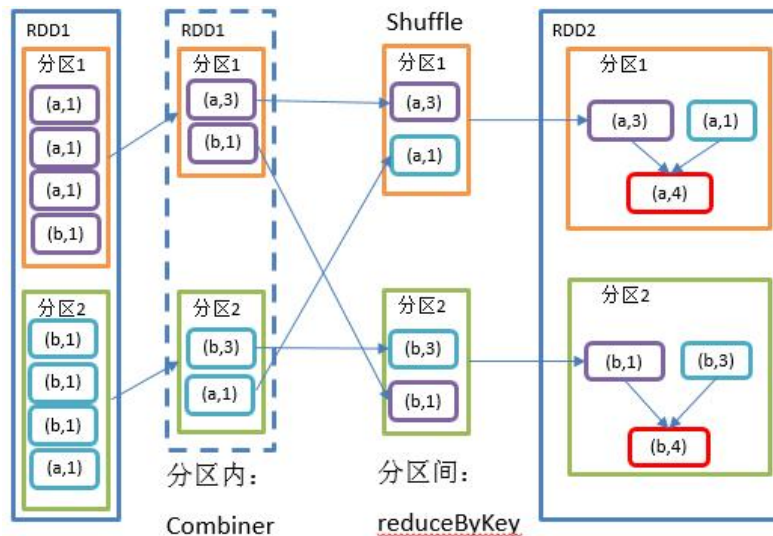
    sc.stop();
  }
}

```

2.3.2.3 reduceByKey()按照 K 聚合 V

1) 功能说明：该操作可以将 RDD[K,V]中的元素按照相同的 K 对 V 进行聚合。其存在多种重载形式，还可以设置新 RDD 的分区数。

2) 需求说明：统计单词出现次数



3) 代码实现：

```

package com.atguigu.keyValue;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.Arrays;

public class Test04 ReduceByKey {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<String> integerJavaRDD =
sc.parallelize(Arrays.asList("hi","hi","hello","spark"),2);
    }
}

```

```
// 统计单词出现次数
JavaPairRDD<String, Integer> pairRDD = integerJavaRDD.mapToPair(new
PairFunction<String, String, Integer>() {
    @Override
    public Tuple2<String, Integer> call(String s) throws Exception {
        return new Tuple2<>(s, 1);
    }
});

// 聚合相同的 key
JavaPairRDD<String, Integer> result = pairRDD.reduceByKey(new Function2<Integer,
Integer, Integer>() {
    @Override
    public Integer call(Integer v1, Integer v2) throws Exception {
        return v1 + v2;
    }
});

result.collect().foreach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
```

2.3.2.4 reduceByKey 和 groupByKey 区别

- 1) reduceByKey: 按照 key 进行聚合, 在 shuffle 之前有 combine (预聚合) 操作, 返回结果是 RDD[K,V]。
- 2) groupByKey: 按照 key 进行分组, 直接进行 shuffle。
- 3) 开发指导: 在不影响业务逻辑的前提下, 优先选用 reduceByKey。求和操作不影响业务逻辑, 求平均值影响业务逻辑。影响业务逻辑时建议先对数据类型进行转换再合并。

```
package com.atguigu.keyValue;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.Arrays;

public class Test06 ReduceByKeyAvg {
    public static void main(String[] args) throws InterruptedException {
        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);
```

```
// 3. 编写代码
JavaPairRDD<String, Integer> javaPairRDD = sc.parallelizePairs(Arrays.asList(new
Tuple2<>("hi", 96), new Tuple2<>("hi", 97), new Tuple2<>("hello", 95), new Tuple2<>("hello",
195)));

// ("hi",96,1))
JavaPairRDD<String, Tuple2<Integer, Integer>> tuple2JavaPairRDD =
javaPairRDD.mapValues(new Function<Integer, Tuple2<Integer, Integer>>() {
    @Override
    public Tuple2<Integer, Integer> call(Integer v1) throws Exception {
        return new Tuple2<>(v1, 1);
    }
});

// 聚合 RDD
JavaPairRDD<String, Tuple2<Integer, Integer>> reduceRDD =
tuple2JavaPairRDD.reduceByKey(new Function2<Tuple2<Integer, Integer>, Tuple2<Integer,
Integer>, Tuple2<Integer, Integer>>() {
    @Override
    public Tuple2<Integer, Integer> call(Tuple2<Integer, Integer> v1, Tuple2<Integer,
Integer> v2) throws Exception {
        return new Tuple2<>(v1._1 + v2._1, v1._2 + v2._2);
    }
});

// 相除
JavaPairRDD<String, Double> result = reduceRDD.mapValues(new
Function<Tuple2<Integer, Integer>, Double>() {
    @Override
    public Double call(Tuple2<Integer, Integer> v1) throws Exception {
        return (new Double(v1._1) / v1._2);
    }
});

result.collect().foreach(System.out::println);

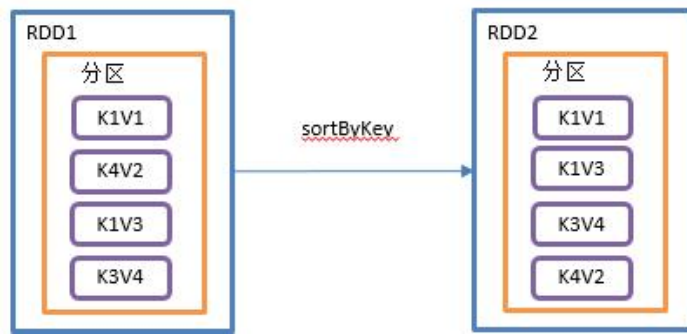
// 4. 关闭 sc
sc.stop();
}
```

2.3.2.5 sortByKey()按照 K 进行排序

1) 功能说明

在一个(K,V)的 RDD 上调用，K 必须实现 Ordered 接口，返回一个按照 key 进行排序的 (K,V)的 RDD。

2) 需求说明：创建一个 pairRDD，按照 key 的正序和倒序进行排序



3) 代码实现:

```
package com.atguigu.keyValue;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaSparkContext;
import scala.Tuple2;

import java.util.Arrays;

public class Test05_SortByKey {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaPairRDD<Integer, String> javaPairRDD = sc.parallelizePairs(Arrays.asList(new
        Tuple2<>(4, "a"), new Tuple2<>(3, "c"), new Tuple2<>(2, "d")));

        // 填写布尔类型选择正序倒序
        JavaPairRDD<Integer, String> pairRDD = javaPairRDD.sortByKey(false);

        pairRDD.collect().forEach(System.out::println);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

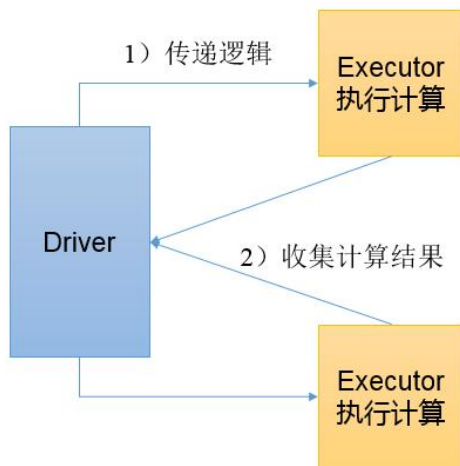
2.4 Action 行动算子

行动算子是触发了整个作业的执行。因为转换算子都是懒加载，并不会立即执行。

1) 创建包名: com.atguigu.action

2.4.1 collect()以数组的形式返回数据集

1) 功能说明: 在驱动程序中，以数组 Array 的形式返回数据集的所有元素。



注意：所有的数据都会被拉取到 Driver 端，慎用。

2) 需求说明：创建一个 RDD，并将 RDD 内容收集到 Driver 端打印

```
package com.atguigu.action;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.Arrays;
import java.util.List;

public class Test01_Collect {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),2);

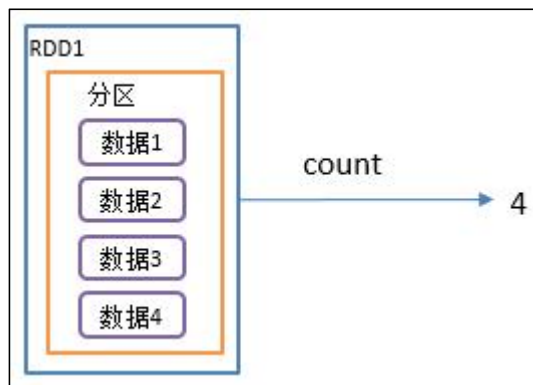
        List<Integer> collect = integerJavaRDD.collect();

        for (Integer integer : collect) {
            System.out.println(integer);
        }

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.4.2 count()返回 RDD 中元素个数

1) 功能说明：返回 RDD 中元素的个数



3) 需求说明: 创建一个 RDD, 统计该 RDD 的条数

```
package com.atguigu.action;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.Arrays;

public class Test02_Count {
    public static void main(String[] args) {

        // 1. 创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4), 2);

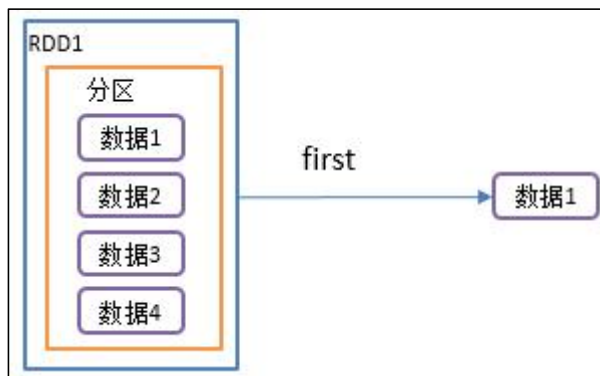
        long count = integerJavaRDD.count();

        System.out.println(count);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.4.3 first()返回 RDD 中的第一个元素

1) 功能说明: 返回 RDD 中的第一个元素



2) 需求说明：创建一个 RDD，返回该 RDD 中的第一个元素

```
package com.atguigu.action;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.Arrays;

public class Test03_First {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),2);

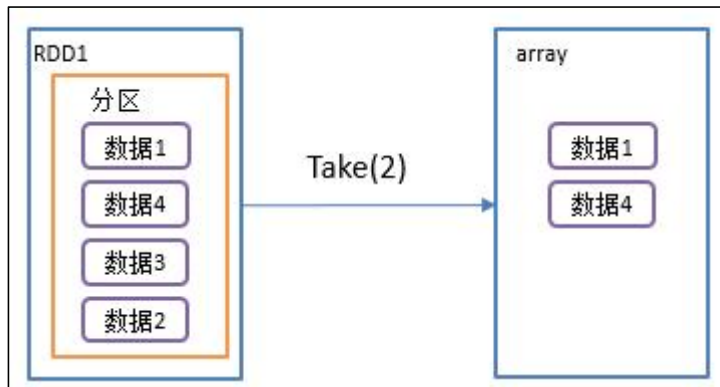
        Integer first = integerJavaRDD.first();

        System.out.println(first);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.4.4 take()返回由 RDD 前 n 个元素组成的数组

1) 功能说明：返回一个由 RDD 的前 n 个元素组成的数组



2) 需求说明：创建一个 RDD，取出前两个元素

```
package com.atguigu.action;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

import java.util.Arrays;
import java.util.List;

public class Test04_Take {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),2);

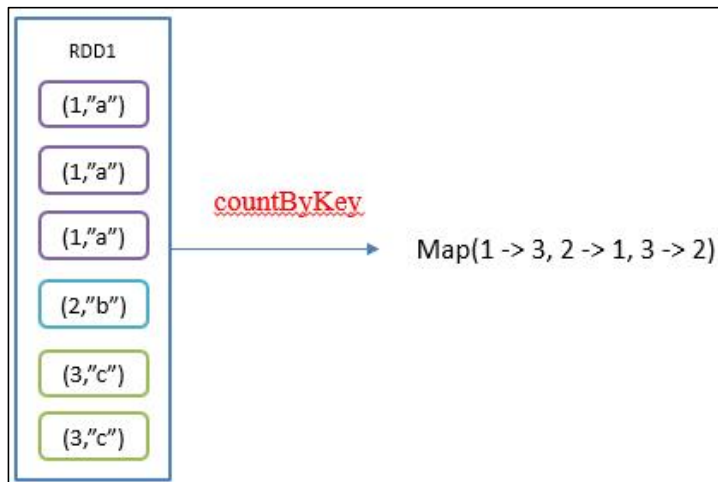
        List<Integer> list = integerJavaRDD.take(3);

        list.forEach(System.out::println);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.4.5 countByKey()统计每种 key 的个数

1) 功能说明：统计每种 key 的个数



2) 需求说明：创建一个 PairRDD，统计每种 key 的个数

```
package com.atguigu.action;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaSparkContext;
import scala.Tuple2;

import java.util.Arrays;
import java.util.Map;

public class Test05_CountByKey {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaPairRDD<String, Integer> pairRDD = sc.parallelizePairs(Arrays.asList(new
        Tuple2<>("a", 8), new Tuple2<>("b", 8), new Tuple2<>("a", 8), new Tuple2<>("d", 8)));

        Map<String, Long> map = pairRDD.countByKey();

        System.out.println(map);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.4.6 save 相关算子

1) saveAsTextFile(path)保存成 Text 文件

功能说明：将数据集的元素以 `textfile` 的形式保存到 HDFS 文件系统或者其他支持的文件系统，对于每个元素，Spark 将会调用 `toString` 方法，将它装换为文件中的文本

2) `saveAsObjectFile(path)` 序列化对象保存到文件

功能说明：用于将 RDD 中的元素序列化成对象，存储到文件中。

3) 代码实现

```
package com.atguigu.action;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import scala.Tuple2;

import java.util.Arrays;

public class Test06_Save {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),2);

        integerJavaRDD.saveAsTextFile("output");

        integerJavaRDD.saveAsObjectFile("output1");

        // 4. 关闭 sc
        sc.stop();
    }
}
```

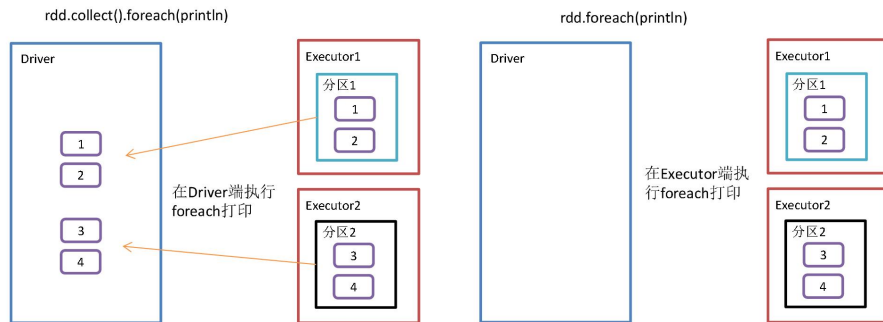
2.4.7 foreach()遍历 RDD 中每一个元素



foreach算子



1) 功能说明：遍历RDD中的每一个元素，并依次应用f函数



让天下没有难学的技术

2) 需求说明：创建一个 RDD，对每个元素进行打印

```
package com.atguigu.action;
```

```
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.VoidFunction;
```

```
import java.util.Arrays;
```

```
public class Test07_Foreach {
    public static void main(String[] args) {
```

```
        // 1. 创建配置对象
```

```
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");
```

```
        // 2. 创建 sparkContext
```

```
        JavaSparkContext sc = new JavaSparkContext(conf);
```

```
        // 3. 编写代码
```

```
        JavaRDD<Integer> integerJavaRDD = sc.parallelize(Arrays.asList(1, 2, 3, 4),4);
```

```
        integerJavaRDD.foreach(new VoidFunction<Integer>() {
```

```
            @Override
```

```
            public void call(Integer integer) throws Exception {
```

```
                System.out.println(integer);
```

```
            }
```

```
        });
```

```
        // 4. 关闭 sc
```

```
        sc.stop();
```

```
    }
}
```

2.4.8 foreachPartition ()遍历 RDD 中每一个分区

```
package com.atguigu.spark.action;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.VoidFunction;

import java.util.Arrays;
import java.util.Iterator;

public class Test08_ForeachPartition {
    public static void main(String[] args) {

        // 1. 创建配置对象
        SparkConf conf = new SparkConf().setAppName("core").setMaster("local[*]");

        // 2. 创建 sc 环境
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> parallelize = sc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6), 2);

        // 多线程一起计算 分区间无序 单个分区有序
        parallelize.foreachPartition(new VoidFunction<Iterator<Integer>>() {
            @Override
            public void call(Iterator<Integer> integerIterator) throws Exception {
                // 一次处理一个分区的数据
                while (integerIterator.hasNext()) {
                    Integer next = integerIterator.next();
                    System.out.println(next);
                }
            }
        });

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2.5 WordCount 案例实操

1) 导入项目依赖

下方的是 scala 语言打包插件 只要使用 scala 语法打包运行到 linux 上面 必须要有

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
    <version>3.3.1</version>
  </dependency>
</dependencies>
```


2.5.1 本地调试

本地 Spark 程序调试需要使用 Local 提交模式，即将本机当做运行环境，Master 和 Worker 都为本地。运行时直接加断点调试即可。如下：

1) 代码实现

```
package com.atguigu;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

public class WordCount {
    public static void main(String[] args) {

        // 1. 创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        // 读取数据
        JavaRDD<String> javaRDD = sc.textFile("input/2.txt");

        // 长字符串切分为单个单词
        JavaRDD<String> flatMapRDD = javaRDD.flatMap(new FlatMapFunction<String, String>() {
            @Override
            public Iterator<String> call(String s) throws Exception {
                List<String> strings = Arrays.asList(s.split(" "));
                return strings.iterator();
            }
        });

        // 转换格式为 (单词, 1)
        JavaPairRDD<String, Integer> pairRDD = flatMapRDD.mapToPair(new PairFunction<String, String, Integer>() {
            @Override
            public Tuple2<String, Integer> call(String s) throws
```

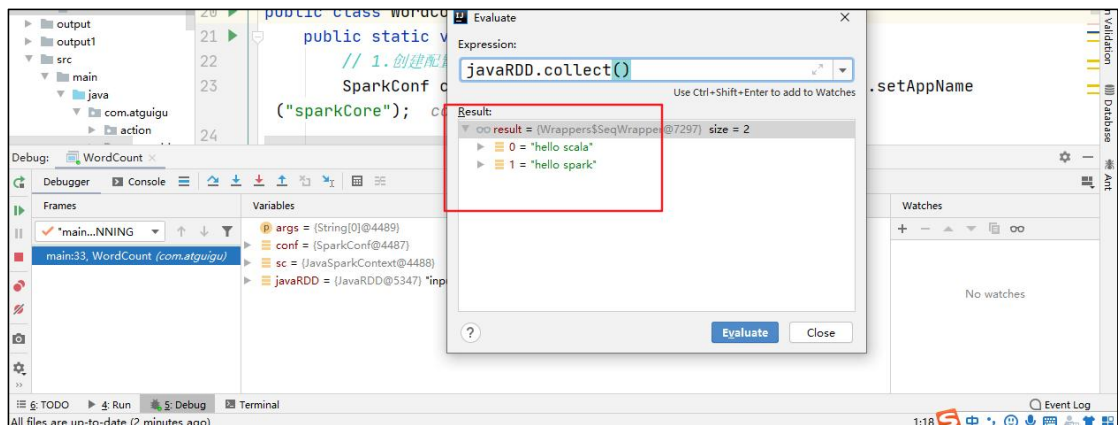
```
Exception {
    return new Tuple2<>(s, 1);
}
});

// 合并相同单词
JavaPairRDD<String, Integer> javaPairRDD =
pairRDD.reduceByKey(new Function2<Integer, Integer, Integer>() {
    @Override
    public Integer call(Integer v1, Integer v2) throws
Exception {
        return v1 + v2;
    }
});

javaPairRDD.collect().foreach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
}
```

2) 调试流程



2.5.2 集群运行

1) 修改代码，修改运行模式，将输出的方法修改为落盘，同时设置可以自定义的传入传出路径

```
package com.atguigu;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.Arrays;
import java.util.Iterator;
```

```
import java.util.List;

public class WordCount {
    public static void main(String[] args) {
        // 1. 创建配置对象
        SparkConf conf = new SparkConf().setMaster("yarn").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        // 读取数据
        JavaRDD<String> javaRDD = sc.textFile(args[0]);

        // 长字符串切分为单个单词
        JavaRDD<String> flatMapRDD = javaRDD.flatMap(new FlatMapFunction<String, String>() {
            @Override
            public Iterator<String> call(String s) throws Exception {
                List<String> strings = Arrays.asList(s.split(" "));
                return strings.iterator();
            }
        });

        // 转换格式为 (单词, 1)
        JavaPairRDD<String, Integer> pairRDD = flatMapRDD.mapToPair(new PairFunction<String, String, Integer>() {
            @Override
            public Tuple2<String, Integer> call(String s) throws Exception {
                return new Tuple2<>(s, 1);
            }
        });

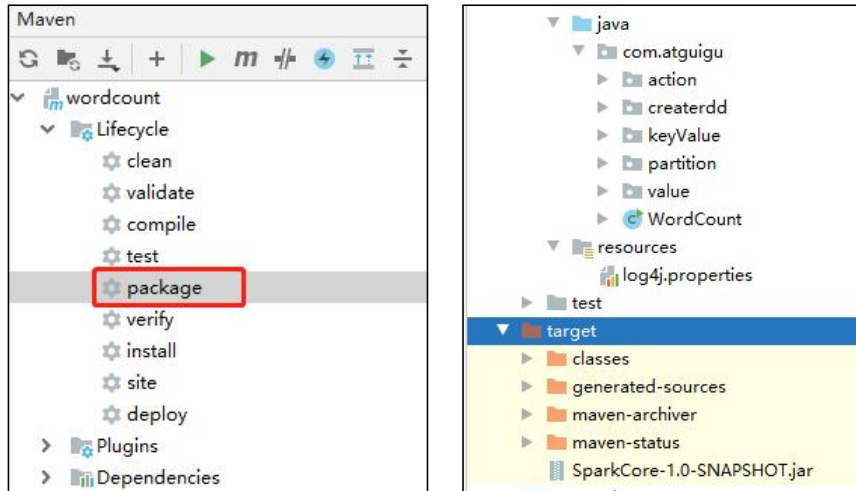
        // 合并相同单词
        JavaPairRDD<String, Integer> javaPairRDD = pairRDD.reduceByKey(new Function2<Integer, Integer, Integer>() {
            @Override
            public Integer call(Integer v1, Integer v2) throws Exception {
                return v1 + v2;
            }
        });

        javaPairRDD.saveAsTextFile(args[1]);

        // 4. 关闭 sc
        sc.stop();
    }
}
```

2) 打包到集群测试

(1) 点击 package 打包，然后，查看打完后的 jar 包



(2) 将 WordCount.jar 上传到/opt/module/spark-yarn 目录

(3) 在 HDFS 上创建，存储输入文件的路径/input

```
[atguigu@hadoop102 spark-yarn]$ hadoop fs -mkdir /input
```

(4) 上传输入文件到/input 路径

```
[atguigu@hadoop102 spark-yarn]$ hadoop fs -put /opt/software/input
```

(5) 执行任务

```
[atguigu@hadoop102 spark-yarn]$ bin/spark-submit \
--class com.atguigu.spark.WordCount \
--master yarn \
./WordCount.jar \
/input \
/output
```

注意：input 和 ouput 都是 HDFS 上的集群路径。

(6) 查询运行结果

```
[atguigu@hadoop102 spark-yarn]$ hadoop fs -cat /output/*
```

2.6 RDD 序列化

在实际开发中我们往往需要自己定义一些对于 RDD 的操作，那么此时需要注意的是，初始化工作是在 Driver 端进行的，而实际运行程序是在 Executor 端进行的，这就涉及到了跨进程通信，是需要序列化的。下面我们看几个例子：

2.6.1 序列化异常

0) 创建包名：com.atguigu.serializable

1) 创建使用的 javaBean: User

```
package com.atguigu.bean;
```

```
import java.io.Serializable;

public class User implements Serializable {
    private String name;
    private Integer age;

    public User(String name, Integer age) {
        this.name = name;
        this.age = age;
    }

    public User() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "User{" +
            "name=" + name + "\" +
            ", age=" + age +
            '\"';
    }
}
```

2) 创建类：Test01_Ser 测试序列化

```
package com.atguigu.serializable;

import com.atguigu.bean.User;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;

import java.util.Arrays;

public class Test01_Ser {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");
```

```
// 2. 创建 sparkContext
JavaSparkContext sc = new JavaSparkContext(conf);

// 3. 编写代码
User zhangsan = new User("zhangsan", 13);
User lisi = new User("lisi", 13);

JavaRDD<User> userJavaRDD = sc.parallelize(Arrays.asList(zhangsan, lisi), 2);

JavaRDD<User> mapRDD = userJavaRDD.map(new Function<User, User>() {
    @Override
    public User call(User v1) throws Exception {
        return new User(v1.getName(), v1.getAge() + 1);
    }
});

mapRDD.collect().foreach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
```

2.6.2 Kryo 序列化框架

参考地址: <https://github.com/EsotericSoftware/kryo>

Java 的序列化能够序列化任何的类。但是比较重，序列化后对象的体积也比较大。

Spark 出于性能的考虑，Spark2.0 开始支持另外一种 Kryo 序列化机制。**Kryo 速度是 Serializable 的 10 倍**。当 RDD 在 Shuffle 数据的时候，简单数据类型、数组和字符串类型已经在 Spark 内部使用 Kryo 来序列化。

```
package com.atguigu.serializable;

import com.atguigu.bean.User;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;

import java.util.Arrays;

public class Test02_Kryo {
    public static void main(String[] args) throws ClassNotFoundException {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore")
            // 替换默认的序列化机制
            .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
            // 注册需要使用 kryo 序列化的自定义类
            .registerKryoClasses(new Class[]{Class.forName("com.atguigu.bean.User")});
    }
}
```

```
// 2. 创建 sparkContext
JavaSparkContext sc = new JavaSparkContext(conf);

// 3. 编写代码
User zhangsan = new User("zhangsan", 13);
User lisi = new User("lisi", 13);

JavaRDD<User> userJavaRDD = sc.parallelize(Arrays.asList(zhangsan, lisi), 2);

JavaRDD<User> mapRDD = userJavaRDD.map(new Function<User, User>() {
    @Override
    public User call(User v1) throws Exception {
        return new User(v1.getName(), v1.getAge() + 1);
    }
});

mapRDD.collect().forEach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
```

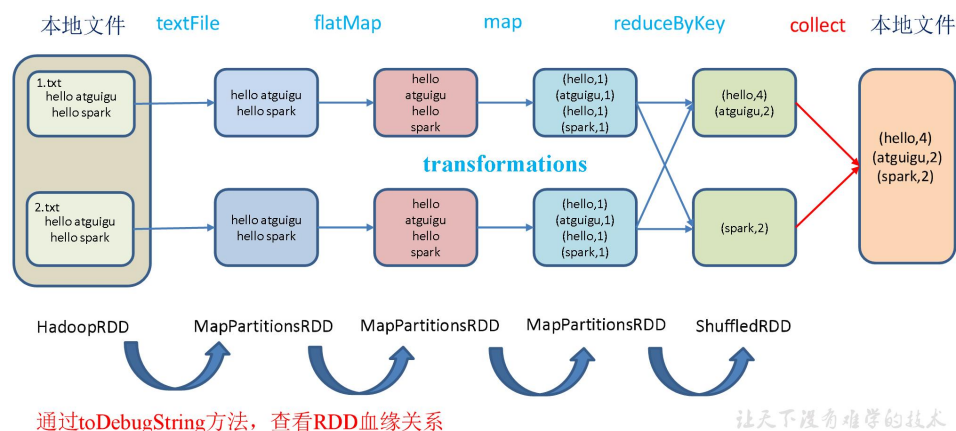
2.7 RDD 依赖关系

2.7.1 查看血缘关系

RDD 只支持粗粒度转换，即在大量记录上执行的单个操作。将创建 RDD 的一系列 Lineage（血统）记录下来，以便恢复丢失的分区。RDD 的 Lineage 会记录 RDD 的元数据信息和转换行为，当该 RDD 的部分分区数据丢失时，它可以根据这些信息来重新运算和恢复丢失的数据分区。



查看血缘关系

让天下没有难学的技术

0) 创建包名：com.atguigu.dependency

1) 代码实现

```
package com.atguigu.dependency;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

public class Test01_Dep {
    public static void main(String[] args) {

        // 1. 创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<String> lineRDD = sc.textFile("input/2.txt");
        System.out.println(lineRDD.toDebugString());
        System.out.println("-----");

        JavaRDD<String> wordRDD = lineRDD.flatMap(new FlatMapFunction<String, String>() {
            @Override
            public Iterator<String> call(String s) throws Exception {
                List<String> stringList = Arrays.asList(s.split(" "));
                return stringList.iterator();
            }
        });
        System.out.println(wordRDD.toDebugString());
        System.out.println("-----");

        JavaPairRDD<String, Integer> tupleRDD = wordRDD.mapToPair(new PairFunction<String,
String, Integer>() {
            @Override
            public Tuple2<String, Integer> call(String s) throws Exception {
                return new Tuple2<>(s, 1);
            }
        });
        System.out.println(tupleRDD.toDebugString());
        System.out.println("-----");

        JavaPairRDD<String, Integer> wordCountRDD = tupleRDD.reduceByKey(new
Function2<Integer, Integer, Integer>() {
            @Override
            public Integer call(Integer v1, Integer v2) throws Exception {
                return v1 + v2;
            }
        });
    }
}
```



```

    }
  });
  System.out.println(wordCountRDD.toDebugString());
  System.out.println("-----");

  // 4. 关闭 sc
  sc.stop();
}
}

```

2) 打印结果

```

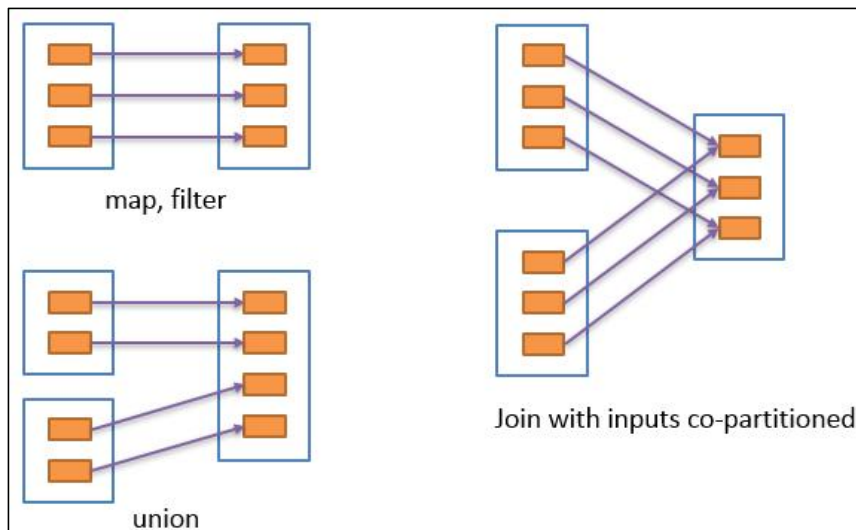
(2) input/2.txt MapPartitionsRDD[1] at textFile at Test01_Dep.java:29 []
| input/2.txt HadoopRDD[0] at textFile at Test01_Dep.java:29 []
(2) MapPartitionsRDD[2] at flatMap at Test01_Dep.java:32 []
| input/2.txt MapPartitionsRDD[1] at textFile at Test01_Dep.java:29 []
| input/2.txt HadoopRDD[0] at textFile at Test01_Dep.java:29 []
(2) MapPartitionsRDD[3] at mapToPair at Test01_Dep.java:42 []
| MapPartitionsRDD[2] at flatMap at Test01_Dep.java:32 []
| input/2.txt MapPartitionsRDD[1] at textFile at Test01_Dep.java:29 []
| input/2.txt HadoopRDD[0] at textFile at Test01_Dep.java:29 []
(2) ShuffledRDD[4] at reduceByKey at Test01_Dep.java:50 []
+-(2) MapPartitionsRDD[3] at mapToPair at Test01_Dep.java:42 []
| MapPartitionsRDD[2] at flatMap at Test01_Dep.java:32 []
| input/2.txt MapPartitionsRDD[1] at textFile at Test01_Dep.java:29 []
| input/2.txt HadoopRDD[0] at textFile at Test01_Dep.java:29 []

```

注意：圆括号中的数字表示 RDD 的并行度，也就是有几个分区

2.7.2 窄依赖

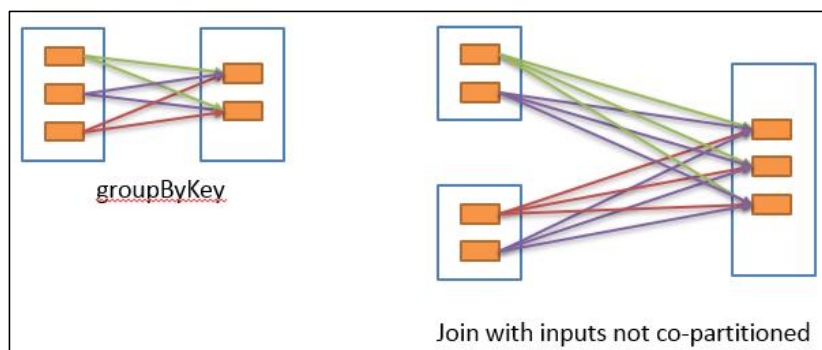
窄依赖表示每一个父 RDD 的 Partition 最多被子 RDD 的一个 Partition 使用（一对一 or 多对一），窄依赖我们形象的比喻为独生子女。



2.7.3 宽依赖

宽依赖表示同一个父 RDD 的 Partition 被多个子 RDD 的 Partition 依赖（只能是一对多），会引起 Shuffle，总结：宽依赖我们形象的比喻为超生。

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网



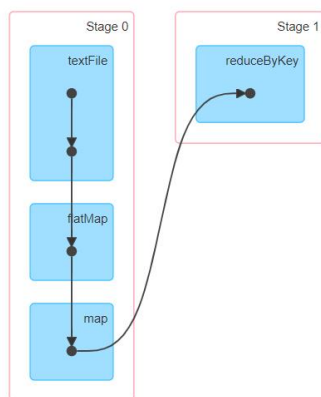
具有宽依赖的 *transformations* 包括：*sort*、*reduceByKey*、*groupByKey*、*join* 和调用 *rePartition* 函数的任何操作。

宽依赖对 Spark 去评估一个 *transformations* 有更加重要的影响，比如对性能的影响。在不影响业务要求的情况下，要尽量避免使用有宽依赖的转换算子，因为有宽依赖，就一定会走 *shuffle*，影响性能。

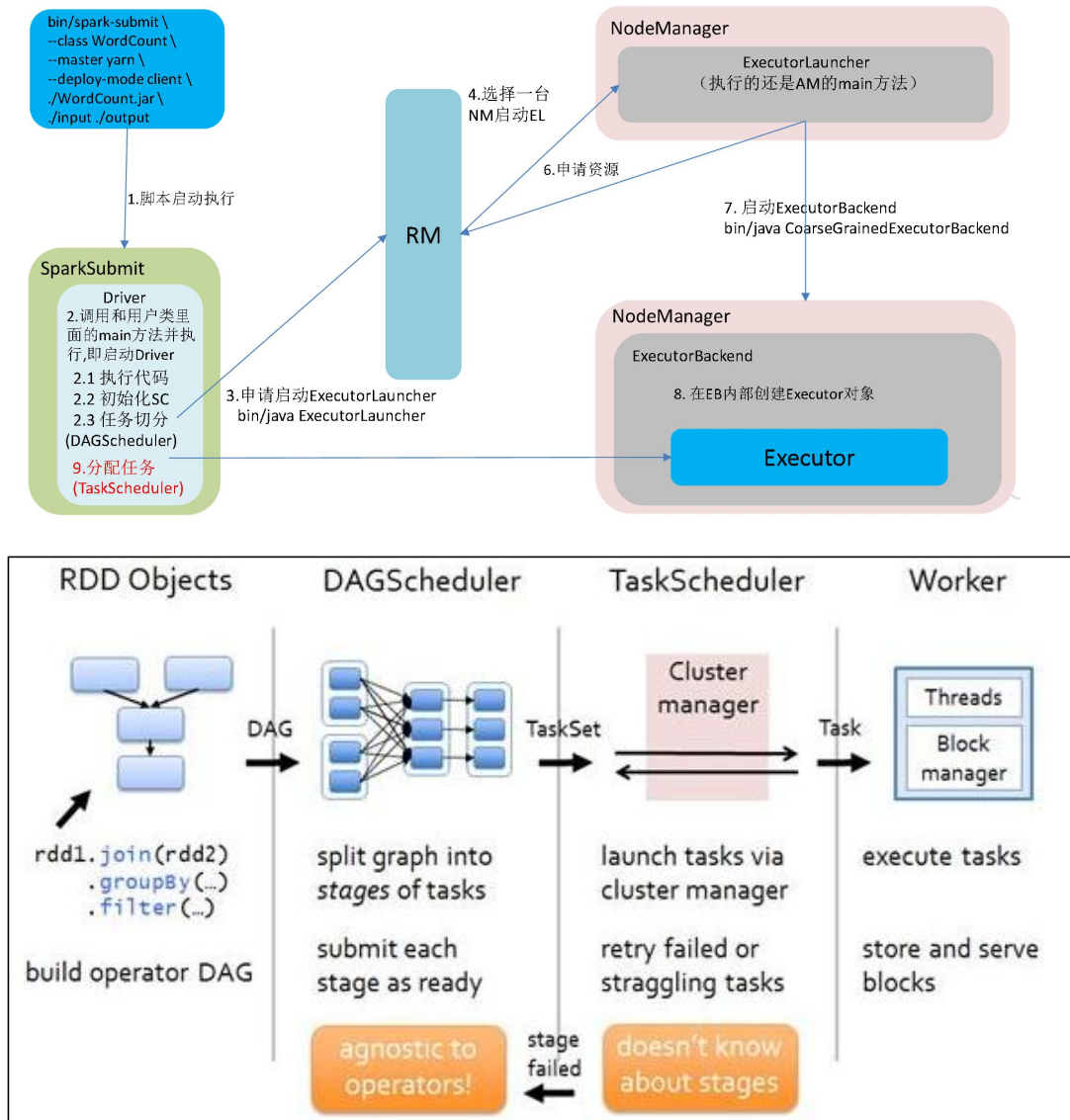
2.7.4 Stage 任务划分

1) DAG 有向无环图

DAG（Directed Acyclic Graph）有向无环图是由点和线组成的拓扑图形，该图形具有方向，不会闭环。例如，DAG 记录了 RDD 的转换过程和任务的阶段。



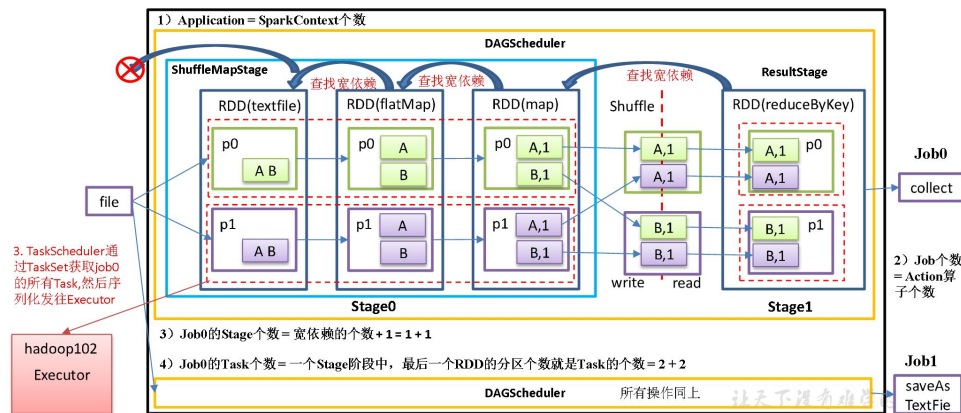
2) 任务运行的整体流程



3) RDD 任务切分中间分为：Application、Job、Stage 和 Task

- (1) Application: 初始化一个 `SparkContext` 即生成一个 Application;
- (2) Job: 一个 Action 算子就会生成一个 Job;
- (3) Stage: Stage 等于宽依赖的个数加 1;
- (4) Task: 一个 Stage 阶段中，最后一个 RDD 的分区个数就是 Task 的个数。

注意： Application->Job->Stage->Task 每一层都是 1 对 n 的关系。



4) 代码实现

```
package com.atguigu.dependency;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

public class Test02_Dep {
    public static void main(String[] args) throws InterruptedException {

        // 1. 创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<String> lineRDD = sc.textFile("input/2.txt");
        System.out.println(lineRDD.toDebugString());
        System.out.println("-----");

        JavaRDD<String> wordRDD = lineRDD.flatMap(new FlatMapFunction<String, String>() {
            @Override
            public Iterator<String> call(String s) throws Exception {
                List<String> stringList = Arrays.asList(s.split(" "));
                return stringList.iterator();
            }
        });
        System.out.println(wordRDD);
    }
}
```

```

System.out.println("-----");

JavaPairRDD<String, Integer> tupleRDD = wordRDD.mapToPair(new PairFunction<String,
String, Integer>() {
    @Override
    public Tuple2<String, Integer> call(String s) throws Exception {
        return new Tuple2<>(s, 1);
    }
});
System.out.println(tupleRDD.toDebugString());
System.out.println("-----");

// 缩减分区
JavaPairRDD<String, Integer> coalesceRDD = tupleRDD.coalesce(1);

JavaPairRDD<String, Integer> wordCountRDD = coalesceRDD.reduceByKey(new
Function2<Integer, Integer, Integer>() {
    @Override
    public Integer call(Integer v1, Integer v2) throws Exception {
        return v1 + v2;
    }
},4);
System.out.println(wordCountRDD.toDebugString());
System.out.println("-----");

wordCountRDD.collect().foreach(System.out::println);
wordCountRDD.collect().foreach(System.out::println);

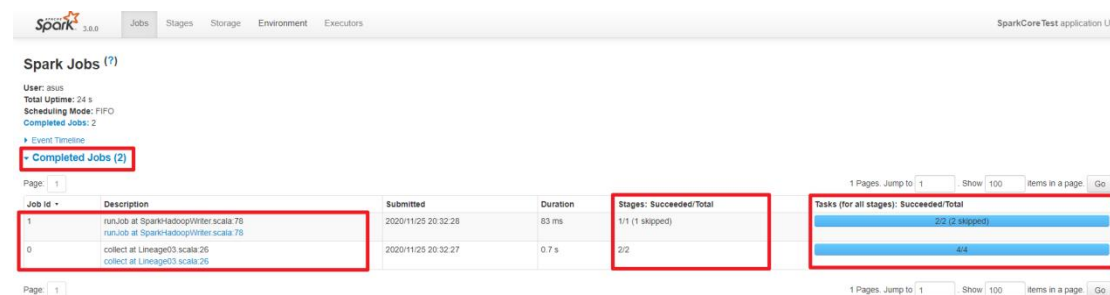
Thread.sleep(600000);

// 4. 关闭 sc
sc.stop();
}
}

```

5) 查看 Job 个数

查看 <http://localhost:4040/jobs/>，发现 Job 有两个。

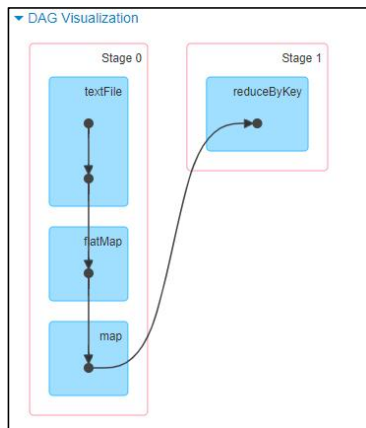


The screenshot shows the Spark Jobs UI. The 'Completed Jobs (2)' section is highlighted with a red box. Below it, a table lists two jobs. The first job (ID 1) is 'runJob at SparkHadoopWriter.scala:78' and the second job (ID 0) is 'collect at Lineage03.scala:26'. Both jobs are in a 'Completed' state. The 'Stages: Succeeded/Total' column shows '1/1 (1 skipped)' for job 1 and '2/2' for job 0. The 'Tasks (for all stages): Succeeded/Total' column shows '2/2 (2 skipped)' for job 1 and '4/4' for job 0.

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1	runJob at SparkHadoopWriter.scala:78	2020/11/25 20:32:28	83 ms	1/1 (1 skipped)	2/2 (2 skipped)
0	collect at Lineage03.scala:26	2020/11/25 20:32:27	0.7 s	2/2	4/4

6) 查看 Stage 个数

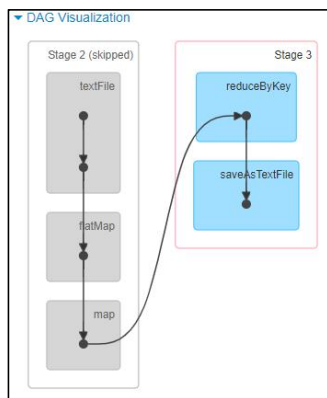
查看 Job0 的 Stage。由于只有 1 个 Shuffle 阶段，所以 Stage 个数为 2。



Completed Stages (2)

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	collect at Lineage03 scala:26	+details 2020/11/25 20:32:26	53 ms	2/2			173.0 B	
0	map at Lineage03 scala:19	+details 2020/11/25 20:32:27	0.5 s	2/2	59.0 B			173.0 B

查看 Job1 的 Stage。由于只有 1 个 Shuffle 阶段，所以 Stage 个数为 2。



Completed Stages (1)

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
3	runJob at SparkHadoopWriter.scala:78	+details 2020/11/25 20:32:28	67 ms	2/2	50.0 B	173.0 B		

Skipped Stages (1)

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
2	map at Lineage03 scala:19	+details Unknown	Unknown	0/2				

7) Task 个数

查看 Job0 的 Stage0 的 Task 个数，2 个。

Tasks (2)

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Input Size / Records	Write Time	Shuffle Write Size / Records	Errors
0	0	0	SUCCESS	PROCESS_LOCAL	driver	LAPTOP-Layne		2020-11-25 12:32:27	0.1 s	96.0 ms	39 B / 2	1.0 ms	111 B / 3	
1	1	0	SUCCESS	PROCESS_LOCAL	driver	LAPTOP-Layne		2020-11-25 12:32:27	0.1 s	96.0 ms	20 B / 1	1.0 ms	62 B / 2	

Showing 1 to 2 of 2 entries

查看 Job0 的 Stage1 的 Task 个数，2 个。

Tasks (2)											
Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Read Size / Records
0	2	0	SUCCESS	NODE_LOCAL	driver	LAPTOP-Layne		2020-11-25 12:32:28	43.0 ms		122 B / 4
1	3	0	SUCCESS	NODE_LOCAL	driver	LAPTOP-Layne		2020-11-25 12:32:28	44.0 ms		51 B / 1

查看 Job1 的 Stage2 的 Task 个数，0 个（2 个跳过 skipped）。

Skipped Stages (1)									
Stage ID	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write	
2	map at Lineage03.scala:19	Unknown	Unknown	0/2					

查看 Job1 的 Stage3 的 Task 个数，2 个。

Tasks (2)											
Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Output Size / Records
0	4	0	SUCCESS	NODE_LOCAL	driver	LAPTOP-Layne		2020-11-25 12:32:28	28.0 ms		36 B / 3
1	5	0	SUCCESS	NODE_LOCAL	driver	LAPTOP-Layne		2020-11-25 12:32:28	28.0 ms		14 B / 1

注意：如果存在 shuffle 过程，系统会自动进行缓存，UI 界面显示 skipped 的部分。

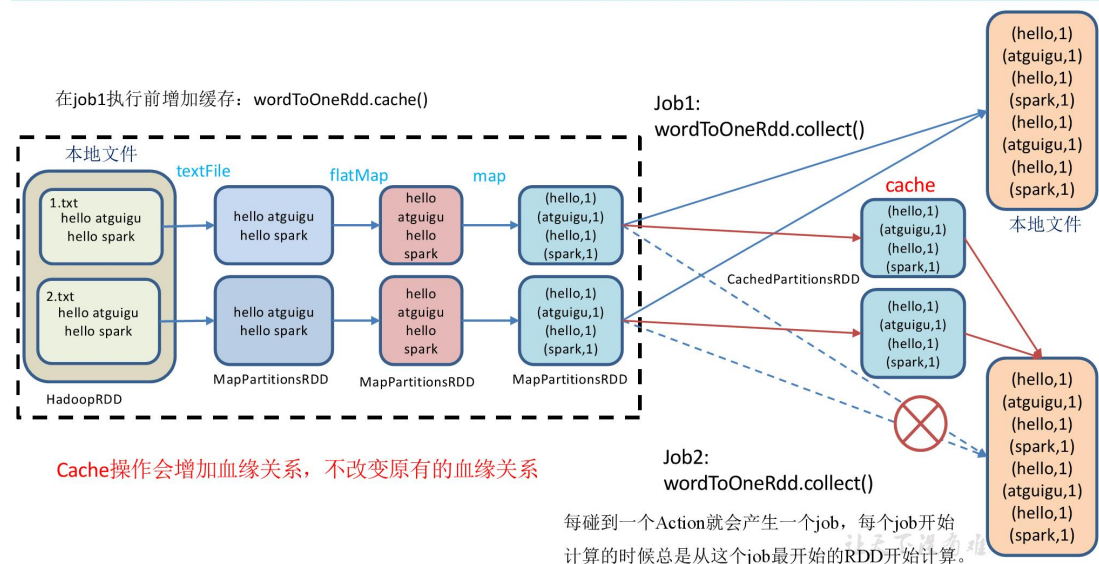
2.8 RDD 持久化

2.8.1 RDD Cache 缓存

RDD 通过 Cache 或者 Persist 方法将前面的计算结果缓存，默认情况下会把数据以序列化的形式缓存在 JVM 的堆内存中。但是并不是这两个方法被调用时立即缓存，而是触发后面的 action 算子时，该 RDD 将会被缓存在计算节点的内存中，并供后面重用。



RDD Cache 缓存



0) 创建包名：com.atguigu.cache

1) 代码实现

```
package com.atguigu.cache;

import org.apache.spark.SparkConf;
```



```
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function;
import scala.Tuple2;

import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

public class Test01_Cache {
    public static void main(String[] args) throws InterruptedException {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<String> lineRDD = sc.textFile("input/2.txt");

        //3.1.业务逻辑
        JavaRDD<String> wordRDD = lineRDD.flatMap(new FlatMapFunction<String, String>() {
            @Override
            public Iterator<String> call(String s) throws Exception {
                List<String> stringList = Arrays.asList(s.split(" "));
                return stringList.iterator();
            }
        });

        JavaRDD<Tuple2<String, Integer>> tuple2JavaRDD = wordRDD.map(new Function<String,
        Tuple2<String, Integer>>() {
            @Override
            public Tuple2<String, Integer> call(String v1) throws Exception {
                System.out.println("*****");
                return new Tuple2<>(v1, 1);
            }
        });

        //3.5 cache 缓存前打印血缘关系
        System.out.println(tuple2JavaRDD.toDebugString());

        //3.4 数据缓存。
        //cache 底层调用的就是 persist 方法,缓存级别默认用的是 MEMORY_ONLY
        tuple2JavaRDD.cache();

        //3.6 persist 方法可以更改存储级别
        // wordToOneRdd.persist(StorageLevel.MEMORY_AND_DISK_2)

        //3.2 触发执行逻辑
        tuple2JavaRDD.collect().foreach(System.out::println);

        //3.5 cache 缓存后打印血缘关系
```



```
//cache 操作会增加血缘关系，不改变原有的血缘关系
System.out.println(tuple2JavaRDD.toDebugString());
System.out.println("=====");

//3.3 再次触发执行逻辑
tuple2JavaRDD.collect().foreach(System.out::println);

Thread.sleep(1000000);

// 4. 关闭 sc
sc.stop();
}
}
```

2) 源码解析

```
mapRdd.cache()
def cache(): this.type = persist()
def persist(): this.type = persist(StorageLevel.MEMORY_ONLY)

object StorageLevel {
  val NONE = new StorageLevel(false, false, false, false)
  val DISK_ONLY = new StorageLevel(true, false, false, false)
  val DISK_ONLY_2 = new StorageLevel(true, false, false, false, 2)
  val MEMORY_ONLY = new StorageLevel(false, true, false, true)
  val MEMORY_ONLY_2 = new StorageLevel(false, true, false, true, 2)
  val MEMORY_ONLY_SER = new StorageLevel(false, true, false, false)
  val MEMORY_ONLY_SER_2 = new StorageLevel(false, true, false, false, 2)
  val MEMORY_AND_DISK = new StorageLevel(true, true, false, true)
  val MEMORY_AND_DISK_2 = new StorageLevel(true, true, false, true, 2)
  val MEMORY_AND_DISK_SER = new StorageLevel(true, true, false, false)
  val MEMORY_AND_DISK_SER_2 = new StorageLevel(true, true, false, false, 2)
  val OFF_HEAP = new StorageLevel(true, true, true, false, 1)
```

注意：默认的存储级别都是仅在内存存储一份。在存储级别的末尾加上“_2”表示持久化的数据存为两份。SER：表示序列化。

级 别	使用的空间	CPU 时间	是否在内存中	是否在磁盘上	备 注
MEMORY_ONLY	高	低	是	否	
MEMORY_ONLY_SER	低	高	是	否	
MEMORY_AND_DISK	高	中等	部分	部分	如果数据在内存中放不下，则溢写到磁盘上
MEMORY_AND_DISK_SER	低	高	部分	部分	如果数据在内存中放不下，则溢写到磁盘上。在内存中存放序列化后的数据
DISK_ONLY	低	高	否	是	

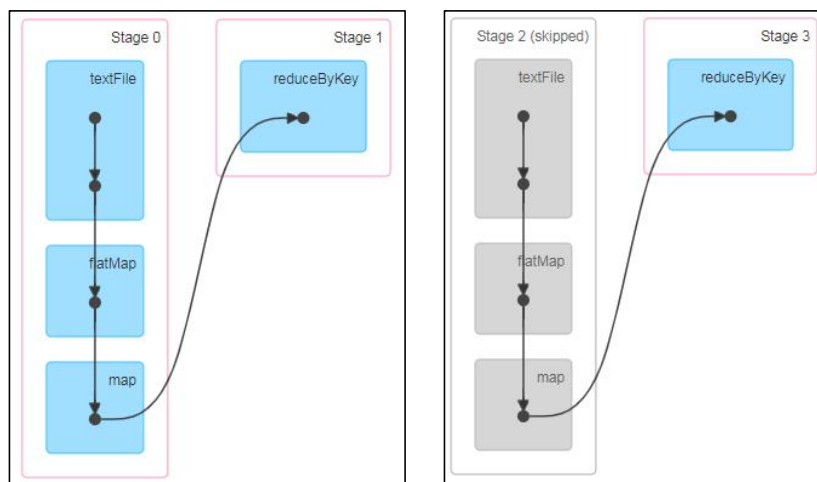
缓存有可能丢失，或者存储于内存的数据由于内存不足而被删除，RDD的缓存容错机制保证了即使缓存丢失也能保证计算的正确执行。通过基于 RDD 的一系列转换，丢失的数据会被重算，由于 RDD 的各个 Partition 是相对独立的，因此只需要计算丢失的部分即可，并不需要重算全部 Partition。

3) 自带缓存算子

Spark 会自动对一些 Shuffle 操作的中间数据做持久化操作（比如：reduceByKey）。这样做的目的是为了当一个节点 Shuffle 失败了避免重新计算整个输入。但是，在实际使用的时候，如果想重用数据，仍然建议调用 persist 或 cache。

查看前面 2.7.4 依赖关系代码的 DAG 图

访问 <http://localhost:4040/jobs/> 页面，查看第一个和第二个 job 的 DAG 图。说明：增加缓存后血缘依赖关系仍然有，但是，第二个 job 取的数据是从缓存中取的。



2.8.2 RDD CheckPoint 检查点

1) 检查点：是通过将 RDD 中间结果写入磁盘。

2) 为什么要做检查点？

由于血缘依赖过长会造成容错成本过高，这样就不如在中间阶段做检查点容错，如果检查点之后有节点出现问题，可以从检查点开始重做血缘，减少了开销。

3) 检查点存储路径：Checkpoint 的数据通常是存储在 HDFS 等容错、高可用的文件系统

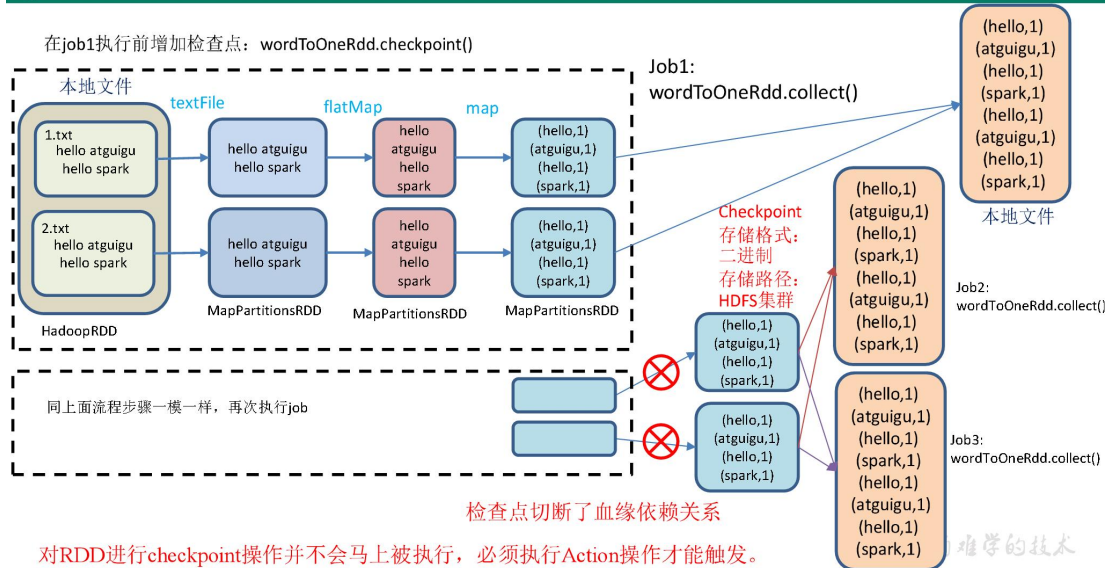
4) 检查点数据存储格式为：二进制的文件

5) 检查点切断血缘：在 Checkpoint 的过程中，该 RDD 的所有依赖于父 RDD 中的信息将全部被移除。

6) 检查点触发时间：对 RDD 进行 Checkpoint 操作并不会马上被执行，必须执行 Action 操作才能触发。但是检查点为了数据安全，会从血缘关系的最开始执行一遍。



Checkpoint检查点



7) 设置检查点步骤

(1) 设置检查点数据存储路径: `sc.setCheckpointDir("./checkpoint1")`

(2) 调用检查点方法: `wordToOneRdd.checkpoint()`

8) 代码实现

```
package com.atguigu.cache;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

public class Test02_CheckPoint {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        sc.setCheckpointDir("ck");

        // 3. 编写代码
        JavaRDD<String> lineRDD = sc.textFile("input/2.txt");

        JavaPairRDD<String, Long> tupleRDD = lineRDD.mapToPair(new PairFunction<String,
String, Long>() {
            @Override
            public Tuple2<String, Long> call(String s) throws Exception {
                return new Tuple2<String, Long>(s, System.currentTimeMillis());
            }
        });
    }
}
```

```

    }
  });

  // 查看血缘关系
  System.out.println(tupleRDD.toDebugString());

  // 增加检查点避免计算两次
  tupleRDD.cache();

  // 进行检查点
  tupleRDD.checkpoint();

  tupleRDD.collect().foreach(System.out::println);

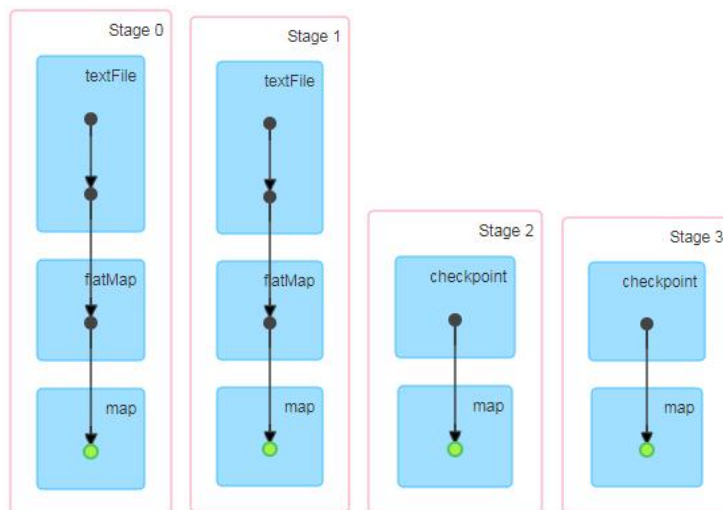
  System.out.println(tupleRDD.toDebugString());
  // 第二次计算
  tupleRDD.collect().foreach(System.out::println);
  // 第三次计算
  tupleRDD.collect().foreach(System.out::println);

  // 4. 关闭 sc
  sc.stop();
}
}

```

9) 执行结果

访问 <http://localhost:4040/jobs/> 页面，查看 4 个 job 的 DAG 图。其中第 2 个图是 checkpoint 的 job 运行 DAG 图。第 3、4 张图说明，**检查点切断了血缘依赖关系**。



(1) 只增加 checkpoint，没有增加 Cache 缓存打印

第 1 个 job 执行完，触发了 checkpoint，第 2 个 job 运行 checkpoint，并把数据存储在检查点上。第 3、4 个 job，数据从检查点上直接读取。

```
(hadoop,1577960215526)
```

```
.....
```

```
(hello,1577960215526)
(hadoop,1577960215609)
. . . . .
(hello,1577960215609)
(hadoop,1577960215609)
. . . . .
(hello,1577960215609)
```

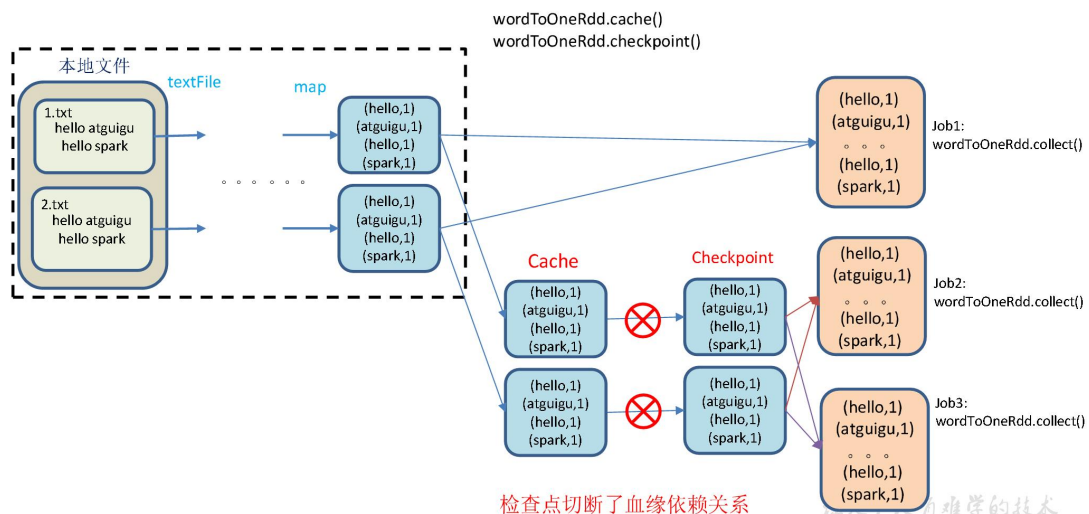
(2) 增加 checkpoint，也增加 Cache 缓存打印

第 1 个 job 执行完，数据就保存到 Cache 里面了，第 2 个 job 运行 checkpoint，直接读取 Cache 里面的数据，并把数据存储在检查点上。第 3、4 个 job，数据从检查点上直接读取。

```
(hadoop,1577960642223)
. . . . .
(hello,1577960642225)
(hadoop,1577960642223)
. . . . .
(hello,1577960642225)
(hadoop,1577960642223)
. . . . .
(hello,1577960642225)
```



CheckPoint检查点+缓存

2.8.3 缓存和检查点区别

(1) Cache 缓存只是将数据保存起来，不切断血缘依赖。Checkpoint 检查点切断血缘依赖。

(2) Cache 缓存的数据通常存储在磁盘、内存等地方，可靠性低。Checkpoint 的数据通常存储在 HDFS 等容错、高可用的文件系统，可靠性高。

(3) 建议对 checkpoint() 的 RDD 使用 Cache 缓存，这样 checkpoint 的 job 只需从 Cache 缓存中读取数据即可，否则需要再从头计算一次 RDD。

(4) 如果使用完了缓存，可以通过 unpersist() 方法释放缓存。

2.8.4 检查点存储到 HDFS 集群

如果检查点数据存储到 HDFS 集群，要注意配置访问集群的用户名。否则会报访问权限异常。

```
package com.atguigu.cache;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

public class Test02_CheckPoint2 {
    public static void main(String[] args) {

        // 修改用户名称
        System.setProperty("HADOOP_USER_NAME", "atguigu");

        // 1. 创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 需要设置路径.需要提前在 HDFS 集群上创建/checkpoint 路径
        sc.setCheckpointDir("hdfs://hadoop102:8020/checkpoint");

        // 3. 编写代码
        JavaRDD<String> lineRDD = sc.textFile("input/2.txt");

        JavaPairRDD<String, Long> tupleRDD = lineRDD.mapToPair(new PairFunction<String,
String, Long>() {
            @Override
            public Tuple2<String, Long> call(String s) throws Exception {
                return new Tuple2<String, Long>(s, System.currentTimeMillis());
            }
        });

        // 查看血缘关系
        System.out.println(tupleRDD.toDebugString());
    }
}
```

```
// 增加检查点避免计算两次
tupleRDD.cache();

// 进行检查点
tupleRDD.checkpoint();

tupleRDD.collect().foreach(System.out::println);

System.out.println(tupleRDD.toDebugString());
// 第二次计算
tupleRDD.collect().foreach(System.out::println);
// 第三次计算
tupleRDD.collect().foreach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
```

2.9 键值对 RDD 数据分区

Spark 目前支持 **Hash 分区**、**Range 分区**和**用户自定义分区**。Hash 分区为当前的默认分区。分区器直接决定了 RDD 中分区的个数、RDD 中每条数据经过 Shuffle 后进入哪个分区和 Reduce 的个数。

1) 注意：

(1) 只有 **Key-Value** 类型的 **pairRDD** 才有分区器，非 **Key-Value** 类型的 RDD 分区的值是 **None**

(2) 每个 RDD 的分区 ID 范围：**0~numPartitions-1**，决定这个值是属于那个分区的。

2) 获取 RDD 分区

(1) 创建包名：**com.atguigu.partitionner**

(2) 代码实现

```
package com.atguigu.partitionner;

import org.apache.spark.Partitioner;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.Optional;
import org.apache.spark.api.java.function.Function2;
import scala.Tuple2;

import java.util.Arrays;

public class Test01_Partitioner {
    public static void main(String[] args) {
```



```
// 1.创建配置对象
SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

// 2. 创建 sparkContext
JavaSparkContext sc = new JavaSparkContext(conf);

// 3. 编写代码
JavaPairRDD<String, Integer> tupleRDD = sc.parallelizePairs(Arrays.asList(new
Tuple2<>("s", 1), new Tuple2<>("a", 3), new Tuple2<>("d", 2)));

// 获取分区器
Optional<Partitioner> partitioner = tupleRDD.partitioner();

System.out.println(partitioner);

JavaPairRDD<String, Integer> reduceByKeyRDD = tupleRDD.reduceByKey(new
Function2<Integer, Integer, Integer>() {
    @Override
    public Integer call(Integer v1, Integer v2) throws Exception {
        return v1 + v2;
    }
});

// 获取分区器
Optional<Partitioner> partitioner1 = reduceByKeyRDD.partitioner();
System.out.println(partitioner1);

// 4. 关闭 sc
sc.stop();
}
```

2.9.1 Hash 分区



Hash分区



HashPartitioner分区的原理：对于给定的key，计算其hashCode，并除以分区的个数取余，如果余数小于0，则用余数+分区的个数（否则加0），最后返回的值就是这个key所属的分区ID。

按照单词首字母的hashCode值%分区个数

A 100万条数据	0分区	A 100万条数据
B 1万条		D 100万条数据
C 1万条		G 100万条数据
D 100万条数据	1分区	B 1万条
E 1万条		E 1万条
F 1万条		
G 100万条数据	2分区	C 1万条
		F 1万条

HashPartitioner分区弊端：可能导致每个分区中数据量的不均匀，极端情况下会导致某些分区拥有RDD的全部数据。

让天下没有难学的技术

2.9.2 Ranger 分区



Ranger分区



RangePartitioner作用：将一定范围内的数映射到某一个分区内，尽量保证每个分区中数据量均匀，而且分区与分区之间是有序的，一个分区中的元素肯定都是比另一个分区内的元素小或者大，但是分区内的元素是不能保证顺序的。简单的说就是将一定范围内的数映射到某一个分区内。

实现过程为：

第一步：先从整个RDD中采用水塘抽样算法，抽取出样本数据，将样本数据排序，计算出每个分区的最大key值，形成一个Array[KEY]类型的数组变量rangeBounds；

第二步：判断key在rangeBounds中所处的范围，给出该key值在下一个RDD中的分区id下标；**该分区器要求RDD中的KEY类型必须是可以排序的**

- | | |
|-------------------------------------|-------|
| 1) 我们假设有100万条数据要分4个区 | ~25 |
| 2) 从100万条中抽100个数 (1,2,3, 100) | 25~50 |
| 3) 对100个数进行排序，然后均匀的分4段 | 50~75 |
| 4) 获取100万条数据，每个值与4个分区的范围比较，放入合适分区 | 75~ |

让天下没有难学的技术

第3章 广播变量

广播变量：**分布式共享只读变量**。

广播变量用来**高效分发较大**的对象。向**所有工作节点**发送一个较大的**只读值**，以供一个或多个 Spark Task 操作使用。比如，如果你的应用需要向所有节点发送一个较大的只读查询表，广播变量用起来会很顺手。在多个 Task 并行操作中使用同一个变量，但是 Spark 会为每个 Task 任务分别发送。

1) 使用广播变量步骤：

(1) 调用 SparkContext.broadcast (广播变量) 创建出一个广播对象，任何可序列化的类型都可以这么实现。

(2) 通过**广播变量.value**，访问该对象的值。

(3) 广播变量只会被发到各个节点一次，作为只读值处理（修改这个值不会影响到别的节点）。

2) 原理说明

```

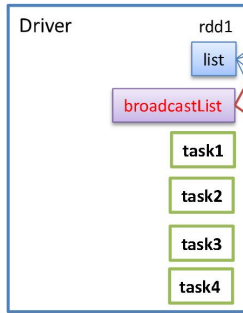
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
JavaRDD<Integer> result = intRDD.filter(new
Function<Integer, Boolean>() {
    @Override
    public Boolean call(Integer v1) throws Exception
    {
        return list.contains(v1);
    }
});

// 创建广播变量
// 只发送一份数据到每一个executor
Broadcast<List<Integer>> broadcast = sc.broadcast(list);

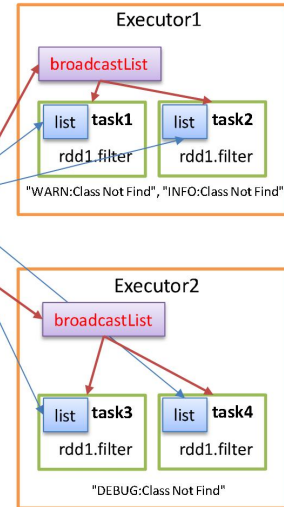
JavaRDD<Integer> result = intRDD.filter(new
Function<Integer, Boolean>() {
    @Override
    public Boolean call(Integer v1) throws Exception {
        return broadcast.value().contains(v1);
    }
});

```

不采用广播变量的方式，list
需要发往每一个task



采用广播变量的方式，
broadcastList需要发往每一个
Executor



让天下没有难学的技术

3) 创建包名: com.atguigu.broadcast

4) 代码实现

```

package com.atguigu.accumulator;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.broadcast.Broadcast;

import java.util.Arrays;
import java.util.List;

public class Test02_Broadcast {
    public static void main(String[] args) {

        // 1.创建配置对象
        SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore");

        // 2. 创建 sparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 3. 编写代码
        JavaRDD<Integer> intRDD = sc.parallelize(Arrays.asList(4, 56, 7, 8, 1, 2));

        // 幸运数字
        List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);

        // 找出幸运数字
        // 每一个 task 都会创建一个 list 浪费内存
        /*
        JavaRDD<Integer> result = intRDD.filter(new Function<Integer, Boolean>() {
            @Override
            public Boolean call(Integer v1) throws Exception {

```

```

        return list.contains(v1);
    }
});
*/

// 创建广播变量
// 只发送一份数据到每一个 executor
Broadcast<List<Integer>> broadcast = sc.broadcast(list);

JavaRDD<Integer> result = intRDD.filter(new Function<Integer, Boolean>() {
    @Override
    public Boolean call(Integer v1) throws Exception {
        return broadcast.value().contains(v1);
    }
});

result.collect().foreach(System.out::println);

// 4. 关闭 sc
sc.stop();
}
}

```

第 4 章 SparkCore 实战

4.1 数据准备

1) 数据格式



数据格式



本项目的数据是采集电商网站的用户行为数据，主要包含用户的4种行为：**搜索、点击、下单和支付**。

日期	用户id	Session id	页面 id	动作时间点	搜索关键字	点击品类id和产品id	下单品类ids和产品id	支付品类ids和产品ids	城市id
2019-07-17	95	26070e87-1ad7-49a3-8fb3-cc741facaddf	37	2019-07-17 00:00:02	手机	-1 -1 null null null null 3			
2019-07-17	95	26070e87-1ad7-49a3-8fb3-cc741facaddf	48	2019-07-17 00:00:10	null	16 98 null null null null 19			
2019-07-17	38	6502cdc9-cf95-4b08-8854-f03a25baa917	24	2019-07-17 00:00:38	null	-1 -1 15,13,5,11,8 99,2 null null 10			
2019-07-17	38	6502cdc9-cf95-4b08-8854-f03a25baa917	22	2019-07-17 00:00:28	null	-1 -1 null null 15,1,20,6,4 15,88,75 9			

- (1) 数据采用_分割字段
- (2) 每一行表示用户的一个行为，所以每一行只能是**四种行为中的一种**。
- (3) 如果点击的品类id和产品id是-1表示**这次不是点击**
- (4) 针对下单行为，**一次可以下单多个产品**，所以品类id和产品id都是多个，**id之间使用逗号分割**。
如果本次不是下单行为，则他们相关数据用**null**来表示。
- (5) 支付行为和**下单行为格式类似**

让天下没有难学的技术

2) 数据详细字段说明

编号	字段名称	字段类型	字段含义
1	date	String	用户点击行为的日期

2	user_id	Long	用户的 ID
3	session_id	String	Session 的 ID
4	page_id	Long	某个页面的 ID
5	action_time	String	动作的时间点
6	search_keyword	String	用户搜索的关键词
7	click_category_id	Long	点击某一个商品品类的 ID
8	click_product_id	Long	某一个商品的 ID
9	order_category_ids	String	一次订单中所有品类的 ID 集合
10	order_product_ids	String	一次订单中所有商品的 ID 集合
11	pay_category_ids	String	一次支付中所有品类的 ID 集合
12	pay_product_ids	String	一次支付中所有商品的 ID 集合
13	city_id	Long	城市 id

4.2 需求：Top10 热门品类



需求说明：品类是指产品的分类，大型电商网站品类分多级，咱们的项目中品类只有一级，不同的公司可能对热门的定义不一样。我们按照每个品类的**点击、下单、支付**的量（次数）来统计热门品类。

鞋 点击数 下单数 支付数

衣服 点击数 下单数 支付数

电脑 点击数 下单数 支付数

例如，**综合排名 = 点击数*20% + 下单数*30% + 支付数*50%**

为了更好的泛用性，当前案例按照点击次数进行排序，如果点击相同，按照下单数，如果下单还是相同，按照支付数。

4.2.1 需求分析

采用样例类的方式实现，聚合算子使用 `reduceByKey`。

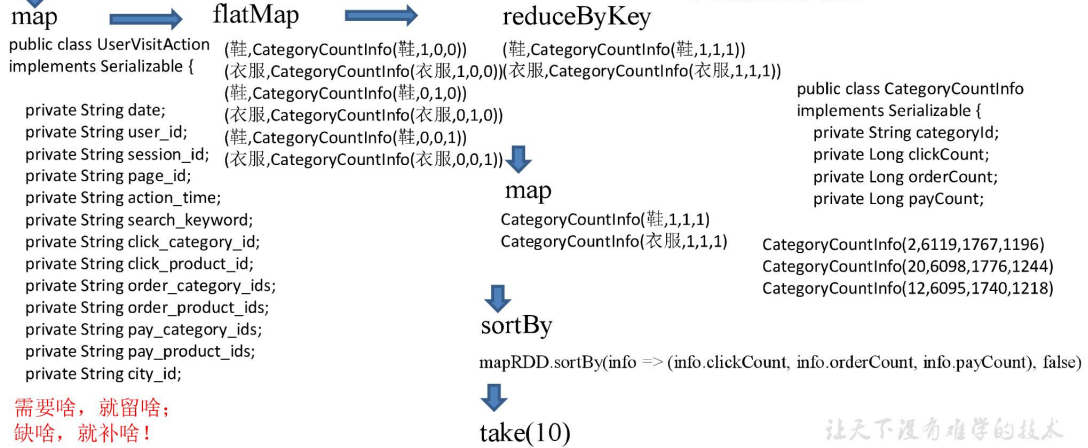


需求分析

点击品类id和产品id 支付品类ids和产品ids 

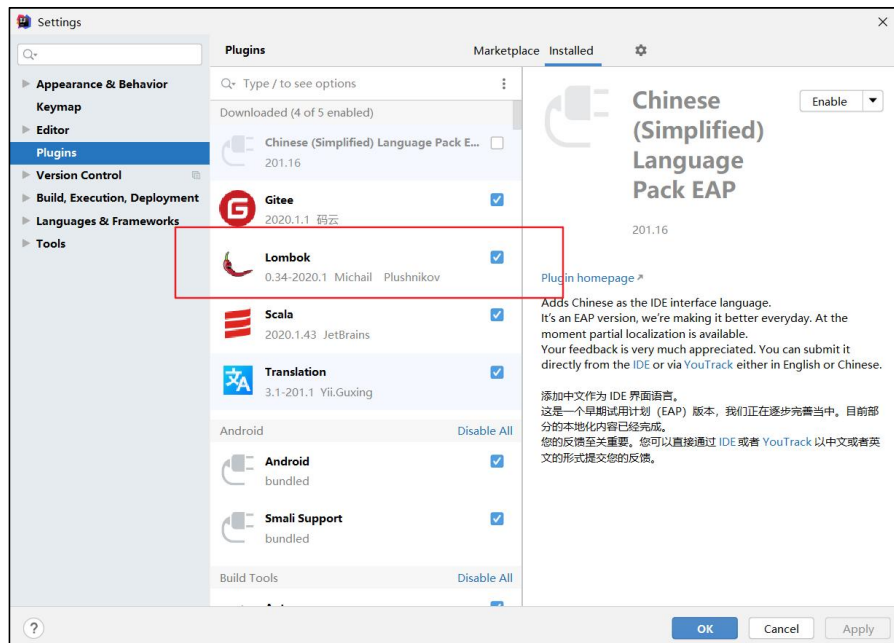
```
2019-07-17_95_26070e87-1ad7-49a3-8fb3-cc741facaddf_37_2019-07-17 00:00:02_手机_-1_-1_null_null_null_3
2019-07-17_95_26070e87-1ad7-49a3-8fb3-cc741facaddf_48_2019-07-17 00:00:10_null_16_98_null_null_null_19
2019-07-17_38_6502cdc9-cf95-4b08-8854-f03a25baa917_24_2019-07-17 00:00:38_null_-1_-1_15,13,5,11,8_99,2_null_null_10
2019-07-17_38_6502cdc9-cf95-4b08-8854-f03a25baa917_22_2019-07-17 00:00:28_null_-1_-1_null_null_15,1,20,6,4_15,88,75_9
```

下单品类ids和产品ids



4.2.2 需求实现

1) 添加 lombok 的插件



2) 添加依赖 lombok 可以省略 getset 代码

```
<dependency>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
<version>1.18.22</version>
</dependency>
```

0) 创建两个存放数据的类

```
package com.atguigu.spark.bean;
```

```
import lombok.Data;

import java.io.Serializable;

@Data
public class UserVisitAction implements Serializable {
    private String date;
    private String user_id;
    private String session_id;
    private String page_id;
    private String action_time;
    private String search_keyword;
    private String click_category_id;
    private String click_product_id;
    private String order_category_ids;
    private String order_product_ids;
    private String pay_category_ids;
    private String pay_product_ids;
    private String city_id;

    public UserVisitAction() {
    }

    public UserVisitAction(String date, String user_id, String session_id, String page_id, String
action_time, String search_keyword, String click_category_id, String click_product_id, String
order_category_ids, String order_product_ids, String pay_category_ids, String pay_product_ids,
String city_id) {
        this.date = date;
        this.user_id = user_id;
        this.session_id = session_id;
        this.page_id = page_id;
        this.action_time = action_time;
        this.search_keyword = search_keyword;
        this.click_category_id = click_category_id;
        this.click_product_id = click_product_id;
        this.order_category_ids = order_category_ids;
        this.order_product_ids = order_product_ids;
        this.pay_category_ids = pay_category_ids;
        this.pay_product_ids = pay_product_ids;
        this.city_id = city_id;
    }
}
```

```
package com.atguigu.spark.bean;

import lombok.Data;

import java.io.Serializable;

@Data
public class CategoryCountInfo implements Serializable, Comparable<CategoryCountInfo> {
    private String categoryId;
    private Long clickCount;
    private Long orderCount;
}
```

```
private Long payCount;

public CategoryCountInfo() {
}

public CategoryCountInfo(String categoryId, Long clickCount, Long orderCount, Long payCount) {
    this.categoryId = categoryId;
    this.clickCount = clickCount;
    this.orderCount = orderCount;
    this.payCount = payCount;
}

@Override
public int compareTo(CategoryCountInfo o) {
    // 小于返回-1,等于返回 0,大于返回 1
    if (this.getClickCount().equals(o.getClickCount())) {
        if (this.getOrderCount().equals(o.getOrderCount())) {
            if (this.getPayCount().equals(o.getPayCount())) {
                return 0;
            } else {
                return this.getPayCount() < o.getPayCount() ? -1 : 1;
            }
        } else {
            return this.getOrderCount() < o.getOrderCount() ? -1 : 1;
        }
    } else {
        return this.getClickCount() < o.getClickCount() ? -1 : 1;
    }
}
}
```

4) 核心业务代码实现

```
package com.atguigu.spark.demo;

import com.atguigu.spark.bean.CategoryCountInfo;
import com.atguigu.spark.bean.UserVisitAction;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import scala.Tuple2;

import java.util.ArrayList;
import java.util.Iterator;

public class Test06_Top10 {
    public static void main(String[] args) throws ClassNotFoundException {

        // 1.创建配置对象
```



```
SparkConf conf = new SparkConf().setMaster("local[*]").setAppName("sparkCore")
// 替换默认的序列化机制
.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
// 注册需要使用 kryo 序列化的自定义类
.registerKryoClasses(new
Class[] {Class.forName("com.atguigu.spark.bean.CategoryCountInfo"), Class.forName("com.atguigu.spark.bean.UserVisitAction")});

// 2. 创建 sparkContext
JavaSparkContext sc = new JavaSparkContext(conf);

// 3. 编写代码
// 分三次进行 wordCount 最后使用 cogroup 合并三组数据
JavaRDD<String> lineRDD = sc.textFile("input/user_visit_action.txt");

// 转换为对象
JavaRDD<UserVisitAction> actionRDD = lineRDD.map(new Function<String,
UserVisitAction>() {
    @Override
    public UserVisitAction call(String v1) throws Exception {
        String[] data = v1.split("_");
        return new UserVisitAction(
            data[0],
            data[1],
            data[2],
            data[3],
            data[4],
            data[5],
            data[6],
            data[7],
            data[8],
            data[9],
            data[10],
            data[11],
            data[12]
        );
    }
});

JavaRDD<CategoryCountInfo> categoryCountInfoJavaRDD = actionRDD.flatMap(new
FlatMapFunction<UserVisitAction, CategoryCountInfo>() {
    @Override
    public Iterator<CategoryCountInfo> call(UserVisitAction userVisitAction) throws
Exception {
        ArrayList<CategoryCountInfo> countInfos = new ArrayList<>();
        if (!userVisitAction.getClick_category_id().equals("-1")) {
            // 当前为点击数据
            countInfos.add(new CategoryCountInfo(userVisitAction.getClick_category_id(), 1L,
0L, 0L));
        } else if (!userVisitAction.getOrder_category_ids().equals("null")) {
            // 当前为订单数据
            String[] orders = userVisitAction.getOrder_category_ids().split(",");
            for (String order : orders) {
                countInfos.add(new CategoryCountInfo(order, 0L, 1L, 0L));
            }
        }
    }
});
```



```
    } else if (!userVisitAction.getPay_category_ids().equals("null")) {
        // 当前为支付数据
        String[] pays = userVisitAction.getPay_category_ids().split(",");
        for (String pay : pays) {
            countInfos.add(new CategoryCountInfo(pay, 0L, 0L, 1L));
        }
    }
    return countInfos.iterator();
}
});

// 合并相同的 id
JavaPairRDD<String, CategoryCountInfo> countInfoJavaPairRDD =
categoryCountInfoJavaRDD.mapToPair(new PairFunction<CategoryCountInfo, String,
CategoryCountInfo>() {
    @Override
    public Tuple2<String, CategoryCountInfo> call(CategoryCountInfo categoryCountInfo)
throws Exception {
        return new Tuple2<>(categoryCountInfo.getCategoryId(), categoryCountInfo);
    }
});

JavaPairRDD<String, CategoryCountInfo> countInfoPairRDD =
countInfoJavaPairRDD.reduceByKey(new Function2<CategoryCountInfo, CategoryCountInfo,
CategoryCountInfo>() {
    @Override
    public CategoryCountInfo call(CategoryCountInfo v1, CategoryCountInfo v2) throws
Exception {
        v1.setClickCount(v1.getClickCount() + v2.getClickCount());
        v1.setOrderCount(v1.getOrderCount() + v2.getOrderCount());
        v1.setPayCount(v1.getPayCount() + v2.getPayCount());
        return v1;
    }
});

JavaRDD<CategoryCountInfo> countInfoJavaRDD = countInfoPairRDD.map(new
Function<Tuple2<String, CategoryCountInfo>, CategoryCountInfo>() {
    @Override
    public CategoryCountInfo call(Tuple2<String, CategoryCountInfo> v1) throws Exception
{
        return v1._2;
    }
});

// CategoryCountInfo 需要能够比较大小
JavaRDD<CategoryCountInfo> result = countInfoJavaRDD.sortBy(new
Function<CategoryCountInfo, CategoryCountInfo>() {
    @Override
    public CategoryCountInfo call(CategoryCountInfo v1) throws Exception {
        return v1;
    }
}, false, 2);

result.collect().forEach(System.out::println);
```

```
// 4. 关闭 sc  
sc.stop();  
}  
}
```